KINGS ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)
Irungattukottai, Sriperumbudur, Chennai – 602117

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
M.E. Computer Science and Engineering (2025–2027)

Project Review: Modules 1 and 2

Code-Mixed Retrieval-Augmented Generation: An Empirical Study of Chunking, Embedding and Query Normalisation for Tamil–English (Tanglish) Queries

Submitted by
SELVIN PAULRAJ K
M.E. Computer Science and Engineering

Guide: _____

Review date: 2 November 2026

Contents

Abstract **4**
Chapter 1: Introduction **5**
1.1 Background 5
1.2 Problem statement 5
1.3 Motivation 5
1.4 Objectives 6
1.5 Scope and limitations 6
1.6 Contributions of Modules 1–2 6
Chapter 2: Literature Survey **8**
2.1 Retrieval-Augmented Generation and retrievers 8
2.2 Hybrid retrieval and rank fusion 8
2.3 Multilingual embedding models and romanisation 8
2.4 Mixed-script and transliterated information retrieval 9
2.5 Transliteration, back-transliteration and romanised language identification 10
2.6 Code-mixed language processing, question answering and RAG 10
2.7 Tamil and Indic retrieval resources, models and RAG component studies 11
2.8 Evaluating RAG and LLM output in multilingual and Indic settings 12
2.9 Datasets 13
2.10 Summary of the literature 13
2.11 Research gap 16
Chapter 3: System Analysis and Design **17**
3.1 Existing system 17
3.2 Proposed system 17
3.3 Existing versus proposed system 17
3.4 System architecture 18
3.5 Module description 18
3.6 Hardware and software 19
Chapter 4: Methodology **20**
4.1 Research questions 20
4.2 Dataset preparation 20
4.3 Query normalisation 20
4.4 Retrievers 21
4.5 Evaluation metrics 22
4.6 Experimental protocol 23
Chapter 5: Module 2 Implementation and Results **24**
5.1 Dataset statistics 24
5.2 Query audit: how code-mixed are the queries? 24
5.3 Main results: raw queries on the Tamil corpus 25
5.4 Effect of query normalisation 26
5.5 Statistical significance 28

5.6 Hybrid fusion can hurt 29
5.7 Secondary setting: English corpus 30
5.8 Latency 31
5.9 Example outputs 31
5.10 Discussion 33

Chapter 6: Conclusion and Future Work 35

6.1 Conclusion 35
6.2 Future modules 35
6.3 Timeline 36

References 37

Abstract

Millions of Tamil speakers type their questions in *Tanglish*: Tamil words written in the Latin alphabet, freely mixed with English ("scholarship ku last date enna?"). Retrieval-Augmented Generation (RAG) systems, which answer questions by first retrieving relevant documents and then generating an answer from them, are almost always built and tested on English queries. This project asks a simple, measurable question: *what happens to each component of a RAG retrieval pipeline when the query is Tanglish instead of English or Tamil script?*

Modules 1 and 2 define the problem, survey related work, design the system, and implement and run a baseline retrieval pipeline. Using the public nlpctx/tamil-english-retrieval dataset, we built a 500-document Tamil-script news corpus with 2,154 queries, each available in four forms: English, Tamil script, romanised Tamil and code-mixed Tanglish. We evaluated five retrieval configurations (BM25 on words, BM25 on character 3-grams, dense retrieval with multilingual-e5-small, and two Reciprocal Rank Fusion hybrids) and two query-normalisation modes that transliterate romanised Tamil into Tamil script (the off-the-shelf ITRANS scheme and a small set of Tanglish rules written for this project). All experiments ran on a 2-core CPU with no GPU.

The results are clear. Tamil-script queries are retrieved well (MRR@10 up to 0.923). Romanised Tamil queries collapse: dense retrieval reaches only MRR@10 = 0.143 (Recall@10 = 0.294), and the best raw configuration only 0.178. Transliterating them to Tamil script with the Tanglish rules raises character-3-gram BM25 to MRR@10 = 0.684 and Recall@10 = 0.812 ($p < 0.001$ under both query-level and document-cluster bootstrap tests) at a cost of about 0.3 ms per query, and the rules beat the ITRANS scheme on every retriever. Code-mixed queries are handled mainly by the dense model (MRR@10 = 0.687), and normalisation does not improve dense retrieval for them ($p = 0.45$). Equal-weight BM25 + dense fusion, a common default in RAG tutorials, *hurts* cross-script retrieval (English queries on Tamil documents: MRR@10 0.788 → 0.710; with a third, character-3-gram list, 0.788 → 0.305). An earlier version of this pipeline overstated that loss (0.788 → 0.187) because zero-score BM25 documents were fed into the fusion; this report gives the corrected numbers. We also found that 22% of the dataset's "code-mixed" queries contain no Tamil word at all, which motivates the human-written Tanglish question set planned for Module 3.

Keywords: Retrieval-Augmented Generation, code-mixing, Tanglish, Tamil, transliteration, BM25, dense retrieval, multilingual embeddings, Reciprocal Rank Fusion.

Chapter 1: Introduction

1.1 Background

A Retrieval-Augmented Generation (RAG) system answers a question in two steps [1]. First, a *retriever* searches a collection of documents and returns the few most likely to contain the answer. Second, a *generator* (a large language model) reads those documents and writes the answer. If the retriever returns the wrong documents, the generator has nothing correct to work with, so retrieval quality sets the ceiling for the whole system.

Retrievers come in two broad families. *Sparse* or lexical retrievers such as BM25 [2] score documents by the query words they share, weighted by how rare those words are. *Dense* retrievers encode the query and each document as vectors with a neural network and rank documents by vector similarity; multilingual dense models such as multilingual-E5 [3], BGE-M3 [4] and LaBSE [5] are trained so that text with the same meaning lands close together across languages. Many RAG systems combine both families with Reciprocal Rank Fusion (RRF) [6], which merges the two ranked lists.

1.2 Problem statement

Tamil has tens of millions of speakers in India, Sri Lanka and abroad, and in chat, search and social media many of them do not type in Tamil script. They type romanised Tamil, often mixed with English words: Tanglish. The same question can arrive in at least four forms:

Form	Example
English	Who is responsible for the Israel-Hamas conflict
Tamil script	இஸ்ரேல் ஹமாஸ் மோதலுக்கு யார் பொறுப்பு
Romanised Tamil	Israel Hamas mothalukku yaar poruppu
Code-mixed (Tanglish)	Israel Hamas conflict ku yaar responsible

Table 1.1: One information need in four query forms (English, romanised and code-mixed forms taken from the evaluation dataset).

The documents a Tamil RAG system searches (government notices, news, college circulars) are usually in Tamil script or English. Romanised Tamil shares neither the script of Tamil documents nor the vocabulary of English documents, and romanisation is unstandardised: the same word can be spelt *kurithu*, *kuritthu* or *kuriththu*. Transliteration-based mixed-script retrieval itself has a history at the Forum for Information Retrieval Evaluation (FIRE), where Tamil appeared in word-level language-labelling subtasks [19], [20]. **What is missing, and what this project addresses, is that no prior study evaluates RAG retrieval components (dense, lexical, fusion) for Tamil–English code-mixed and romanised queries.**

1.3 Motivation

1. **Real users.** Students, farmers and citizens in Tamil Nadu routinely ask questions in Tanglish. A college help-desk bot or an e-governance assistant that works only for English or Tamil-script queries fails the users it is meant to serve.
2. **Hidden failure.** RAG tutorials recommend hybrid BM25 + dense retrieval as a safe default. Whether that advice holds across scripts has not been checked for Tamil.

3. **Cheap fixes may exist.** If a simple, fast normalisation step recovers most of the lost accuracy, it can be added to any existing RAG system without retraining a model.
4. **Research gap.** Code-mixed NLP for Dravidian languages has focused on sentiment and offensive-language classification [7], [8]. The FIRE transliterated-search and mixed-script IR tracks (2013–2016) ran their retrieval subtasks on Hindi and Bengali, and included Tamil only in word-level language labelling [19], [20]; later code-mixed retrieval tasks used Bengali–English [9], [10], and code-switching retrieval benchmarks used Chinese and Japanese [11]. No prior study evaluates RAG retrieval components (dense, lexical, fusion) for Tamil–English code-mixed and romanised queries.

1.4 Objectives

The overall objective is an empirical study, with a reusable benchmark and open code, of chunking, embedding and query normalisation for Tanglish queries in RAG. The objectives for the full project are:

1. **O1.** Build an evaluation set in which the same information need is expressed as English, Tamil-script, romanised and code-mixed queries over a Tamil document collection. *(Modules 2 and 3.)*
2. **O2.** Measure how far retrieval quality drops for each query form, for lexical, dense and hybrid retrievers. *(Module 2, done.)*
3. **O3.** Design and evaluate lightweight query normalisation (transliteration to Tamil script) and measure the accuracy it recovers and the latency it costs. *(Module 2, done for two methods.)*
4. **O4.** Evaluate Tamil-aware document chunking against standard chunkers on longer documents. *(Module 3.)*
5. **O5.** Measure end-to-end answer quality of a RAG system (retrieval plus LLM generation) for each query form. *(Module 4.)*
6. **O6.** Publish the findings, benchmark and code. *(Module 5.)*

1.5 Scope and limitations

In scope: text queries in four forms; Tamil-script and English documents; BM25, character n-gram BM25, dense multilingual embeddings and RRF fusion; rule-based and scheme-based transliteration; CPU-only execution so the work is reproducible on a student laptop; standard IR metrics with confidence intervals.

Out of scope for Modules 1–2: speech input, neural transliteration models (IndicXlit [22] is planned as a Module 3 baseline), fine-tuning embedding models, re-rankers, and answer generation (Module 4). Chunking is designed in Module 1 but not varied in Module 2, because the documents in the current dataset are short (median 326 characters), so each document is already a single chunk.

Known limitation of the current data: every query in the dataset was generated by an LLM, not written by a person. Section 5.2 shows this matters, and Module 3 addresses it.

1.6 Contributions of Modules 1–2

1. A problem definition and a design for a factorial study of RAG components under four query forms.

2. A literature survey of 19 relevant works, including the FIRE mixed-script IR tracks, with a comparison table and a stated research gap.
3. A working, CPU-only retrieval pipeline that reuses the retriever and metric design of the author's ChunkLab project [12].
4. A Tanglish-to-Tamil-script rule transliterator and a comparison against the ITRANS scheme.
5. Real results for 2,154 queries × 8 query sets × 5 retrievers on a Tamil corpus, plus 4 query sets × 5 retrievers on an English corpus, with query-level and document-cluster bootstrap tests and latency.
6. An audit showing that a public "code-mixed" Tamil query set is substantially English, which shapes the Module 3 data plan.

Chapter 2: Literature Survey

2.1 Retrieval-Augmented Generation and retrievers

Lewis et al. [1] introduced RAG, which combines a dense passage retriever with a sequence-to-sequence generator, and showed that retrieval grounds generation in external knowledge. BM25 [2], a probabilistic term-weighting function, remains a strong baseline and is the lexical retriever in most RAG stacks.

Retrieval quality depends on what the retriever receives as a query. Ma et al. [65] add an explicit *rewrite* step before retrieval (Rewrite-Retrieve-Read), done by a prompted LLM or a small trained rewriter. HyDE [66] instead has an LLM write a hypothetical answer document and retrieves with that document's embedding. Both methods were developed for English. They are the natural templates for the LLM query normaliser planned for Module 3, which rewrites a Tanglish query into Tamil script or English before retrieval.

2.2 Hybrid retrieval and rank fusion

Cormack et al. [6] proposed Reciprocal Rank Fusion (RRF), which sums 1/(k + rank) across ranked lists with k = 60. It needs no score calibration and is the usual way to build a "hybrid" retriever. Later analyses are more cautious:

- Bruch et al. [63] found RRF to be sensitive to its parameters. A tuned convex combination of normalised lexical and dense scores outperformed RRF both in and out of domain and needed only a small set of training examples.
- Chauhan and Agarwal [64] measured fusion query by query on seven BEIR collections with five dense encoders, SPLADE and three fusion functions. They treat "do not fuse" as a valid outcome. Even the best fusion ranked relevant documents below the better of its two components on 13–48% of queries. RRF carried the largest loss, the "fusion tax", wherever the dense retriever was clearly stronger than BM25.

In code-mixed IR the picture differs. In the FIRE 2025 CMIR track, fusion and hybrid systems consistently beat standalone models [10], and a BM25 + SBERT re-ranking system improved its BM25 baseline by 83% [30]. In both cases queries and documents share the Roman script, so both lists carry signal. Chapter 5 studies the opposite case: English or romanised queries against Tamil-script documents, where the lexical list is nearly uninformative. This is the condition under which [64] predicts that RRF hurts.

2.3 Multilingual embedding models and romanisation

LaBSE [5] learns language-agnostic sentence embeddings for 109 languages from translation pairs. Multilingual-E5 [3] is a family of text-embedding models trained with weakly supervised contrastive pre-training followed by supervised fine-tuning. It expects "query:" and "passage:" prefixes. BGE-M3 [4] supports dense, sparse and multi-vector retrieval in one model across more than 100 languages. SONAR [58] is a 200-language, fixed-size sentence space with text and speech encoders. It outperforms LASER3 and LaBSE on the xsim and xsim++ similarity-search tasks, and the Large Concept Model [59] builds a language model directly in this space.

These models are trained mainly on native-script text. Several recent studies show what happens with romanised input:

- Chari et al. [28] showed that multilingual dense retrievers, BGE-M3 included, "rapidly deteriorate" on Latinised queries (Greeklish, Arabizi and similar), creating a "script gap". Fine-tuning on an even mix of native and Latinised queries closed most of it. Their languages include Chinese, Russian, Arabic, Greek and Persian, but no Indic language.
- Verma et al. [61] found that language-associated units in multilingual LMs are conditioned on orthography: "romanization induces near-disjoint representations that align with neither native-script inputs nor English". This is a likely mechanism for the romanised-query collapse in Section 5.3.
- RomanSetu [60] reports the opposite effect for an LLM continually pre-trained on romanised text. There, romanisation cut token fertility by 2–4× and aligned *better* with English. Romanised text is therefore a problem for models that never saw it, not an inherently poor input.
- Zhu et al. [62] interpolated the embeddings of parallel queries in BGE-M3. A mixed query beat the best single-language query in 88 of 105 cases, while English queries worked best against indices that contain English.

2.4 Mixed-script and transliterated information retrieval

FIRE transliterated search and MSIR. Gupta et al. [21] formalised mixed-script information retrieval, where queries and documents in one language appear in both the native and the Roman script. They estimated from Bing logs that 6% of unique queries from India contained romanised Hindi words. They expanded queries with cross-script term equivalents learned by a deep autoencoder, raising MRR from 0.686 to 0.874 on Hindi song-lyrics retrieval (FIRE 2013 collection, 62,888 documents, 25 queries).

The FIRE Transliterated Search track began in 2013 [24], [19]: - 2013: word-level language labelling with back-transliteration for Hindi, Bangla and Gujarati, plus retrieval of Roman-script Bollywood lyrics queries (25 runs from five teams) [24]. - 2014: the labelling subtask added Tamil, Kannada and Malayalam [19]. One 2014 system normalised transliterated character sequences with rules at indexing time and used prefix-matched fuzzy query terms, gaining about 56% over a query-likelihood baseline [25]. - 2015: renamed Mixed Script IR. It labelled words in code-mixed text for eight languages including Tamil, while its retrieval subtask remained Hindi only (song lyrics, movie reviews and astrology; 25 test queries) [20]. One 2015 system transliterated every Roman-script word to Devanagari and indexed character 2- to 6-grams with vowel signs removed, which gave the best nDCG@10 among its runs [20]. This is a direct precedent for our transliteration plus character-n-gram design. - 2016: Bengali–English question classification and Hindi–English tweet retrieval, with a best MAP of only 0.0377 [19]. That tweet retrieval task built on the CMIR corpus of Chakma and Das [26].

In short, transliteration-based mixed-script IR, including rule-based normalisation, is an established FIRE topic. We do not claim the idea as new. However, its retrieval tasks never covered Tamil and predate dense retrievers and RAG.

FIRE CMIR. The FIRE Code-Mixed Information Retrieval (CMIR) track [9], [10] is the closest recent retrieval benchmark. Its 2024 edition used 50 Bengali–English code-mixed queries over 107,900 social-media comments. The best system, built on GPT-3.5 Turbo, reached

MAP 0.70 [9]; other participants combined prompting with ranking models [29]. The 2025 edition added native-script Bengali [10]. The language is Bengali, the documents are social-media comments, and RAG components are not studied.

Code-switching IR beyond India. Zeng et al. [11] built a human-annotated code-switching retrieval benchmark (CSR-L) and an 11-task CS-MTEB suite for Chinese and Japanese mixed with English. Code-switching degraded statistical, dense and late-interaction retrievers, with drops of up to 27%, and vocabulary expansion did not close the gap. This supports our hypothesis but covers neither Tamil nor romanised script. The paper is now published in *Findings of ACL 2026*.

2.5 Transliteration, back-transliteration and romanised language identification

Transliteration resources. - Aksharantar [22] is a 26-million-pair transliteration corpus for 21 Indic languages. IndicXlit, trained on it, is an open neural model for romanised-to-native transliteration and the natural neural baseline for our rule-based normaliser. - The Dakshina dataset [31] provides native and Latin-script parallel sentences and attested romanisation lexicons for 12 South Asian languages, Tamil included. - Kirov et al. [32] used context, including fine-tuned pretrained models trained on simulated parallel data, to substantially improve full-sentence Latin-to-native transliteration on all twelve Dakshina languages. - Riyadh and Kondrak [27] combined language identification, back-transliteration and sequence prediction to deromanise code-mixed Bengali and Hindi text. - For Tamil specifically, TAMZHI [35] targets “shorthand” romanised Tamil in which vowels are dropped. The difficulty comes from Tamil’s twelve vowels against the Roman alphabet’s five, which is the same ambiguity our Tanglish rules face. - IndicCMix [23] is a recent AI4Bharat dataset with about 104,800 Tamil rows. Each row gives an English sentence, a native-script code-mixed version, a full Tamil translation and a casual romanised version. It has no accompanying paper or retrieval relevance judgements.

Romanised language identification. IndicLID [33] is the first language identifier for romanised text in the 22 scheduled Indian languages. Training on synthetic samples with natural spelling variation raised romanised-text F1 on the Bhasha-Abhijnaanam test set from 74.7% to 88.2% [34]. Code-mixed web queries often carry entities in another language, and gazetteer-enhanced NER handles them (F1 = 68% on the EMBER code-mixed query set) [77]. Both lines bear on the script-gated router and the entity-safe language ID planned for Module 3.

None of these transliteration works measures the effect of normalisation on *retrieval*. They report character or word accuracy only.

2.6 Code-mixed language processing, question answering and RAG

Code-mixed QA. Chandu et al. [13] crowd-sourced a code-mixed factoid question-answering dataset with 1,694 Hinglish, 2,848 Tamlish (Tamil–English) and 1,391 Tenglish questions. It is the closest earlier Tamil–English resource, but it targets factoid QA, not RAG retrieval components. In a related Hindi–English knowledge-base QA study, back-transliterated code-mixed questions beat lexically and sentence-translated questions by 5% and 35% in accuracy [36]. This is early evidence that transliteration can be a better front-end than translation. GLUECoS [37] added code-switched QA and NLI to a Hindi–English and Spanish–English benchmark, and Winata et al. [38] survey the field.

Code-mixed classification in Tamil. Chakravarthi et al. [7] built a Tamil–English (Tanglish) sentiment corpus from YouTube comments. The Dravidian-CodeMix track at FIRE 2020 [8], continued in 2021, used it for sentiment-classification shared tasks. The DravidianLangTech workshops continue this line: their 2025 and 2026 shared tasks cover abuse, sentiment, memes and speech, but not retrieval. These works show that Tanglish is common online, but they are classification tasks.

LLMs on code-mixed and romanised input. - Multilingual LLMs underperform much smaller fine-tuned models on code-switched sentiment, translation, summarisation and language identification [39]. - CodeMixQA [40] adds human-annotated code-switched QA in 16 language-pair variants, in both native and transliterated forms, and finds persistent reasoning and generation failures. - Indi-RomCoM [41] evaluates instruction following on romanised code-mixed Hindi, Bengali, Gujarati and Tamil at three mixing levels. Performance falls as code-mixing density rises. - TanglishMedBench [57] provides 700 romanised Tanglish medical questions with English equivalents and expert annotations. It has no document collection or relevance judgements, so it cannot evaluate retrieval.

Code-mixed RAG (2025–2026). - Sharma et al. [43] retrieved English documents for informal Hinglish queries. A fine-tuned multilingual embedder reached 96.3% accuracy on the strict task against 76.7% for the best zero-shot model. - Gaikwad and Gaikwad [42] added lightweight code-mixed query normalisation and an evidence-constrained abstention layer to a RAG pipeline. Hallucination fell most for Hindi–English code-mixed queries. - OneVoice [44] is a deployed college-support RAG chatbot. It detects code-mixing at the Unicode level and lists Hinglish, Telugish and Tamlish among its variants, but reports no retrieval ablation. Its domain (student queries about college deadlines) is the same as the Module 3 collection. - Similar studies exist for Tagalog–English RAG [45].

A caution on normalisation. In a factorial ablation of a Hinglish banking NLU pipeline, Agarwal [46] found that a hand-built normaliser helped a base encoder but became redundant after noisy-query fine-tuning. On native-reviewed Tamil and Telugu probes, its effect was “unstable in sign”. This agrees with our Module 2 finding that normalisation helps lexical retrieval for romanised queries but not dense retrieval for code-mixed queries (Section 5.4), and it argues for testing normalisation together with fine-tuning in Module 3.

2.7 Tamil and Indic retrieval resources, models and RAG component studies

Corpora, translation and models. - IndicCorp v2 (20.9 billion tokens across 24 languages), the IndicXTREME benchmark and IndicBERT v2 [47] form the base of most Indic NLU work. The nlpctx documents [18] derive from IndicCorpV2 news text. - IndicTrans2 [48] provides open machine translation for all 22 scheduled languages. It produced the English versions in [18]. - Tamil-specific generators include Tamil-Llama [49], which extends LLaMA-2 with 16,000 Tamil tokens, and Agaram 2.5B [50]. Agaram is a Tamil-first model trained from scratch that converses in Tamil script, romanised Tanglish and English. Its tokeniser needs 1.55 tokens per Tamil word against 9.44 for Qwen2.5. - Sarvam-1 [51] is a 2-billion-parameter model for ten Indic languages including Tamil. These are candidates for the Module 4 generator.

Retrieval and RAG benchmarks. - MIRACL [55], the main multilingual ad hoc retrieval benchmark (18 languages, 726k judgements), includes Telugu but not Tamil. - INDIC-MARCO and Indic-ColBERT [52] cover 11 Indian languages including Tamil. They were

built by machine-translating MS MARCO. - IndicMSMarco, part of IndicRAGSuite [53], has 1,000 manually translated MS MARCO queries in 13 languages, Tamil included, for evaluating both retrieval and generation. - Indic-QA [54] evaluates context-grounded QA in 11 Indic languages and finds that Translate-Test beats direct multilingual LLMs in low-resource settings. - MURMUR [56] tests cross-lingual multimodal RAG in Tamil and Yoruba. Its XM-RAG baseline reaches 38.1 answer accuracy, far below English results. Every one of these benchmarks uses **native-script** Tamil queries.

Component studies. - Balasubramaniyam and Madushanka [14] compared query translation with cross-lingual embeddings (LaBSE, multilingual-E5, BGE-M3) for Sinhala and Tamil queries over English e-government content, using 500 human-verified question–answer pairs. BGE-M3 reached Recall@15 of 95.6% for Tamil–English. Their Tamil queries are in native script only; romanised and code-mixed queries are not studied. - Singh [15] ran a controlled factorial study of 3 parsers × 3 chunking strategies × 5 dense embedding models, plus BM25, on 800 questions over Indian government regulatory documents, with mixed-effects models and bootstrap intervals. It is the methodological model for this project, but it is English-only.

Chunking. - Chhoun et al. [16] compared four chunking strategies (recursive, Khmer-aware, sentence-based, LLM-based) for Khmer agricultural RAG with BGE-M3. They found 300-character recursive chunking best, which motivates our Tamil-aware chunker in Module 3, although they used only 18 question–answer pairs. - More general studies show that the choice of retrieval unit significantly affects both retrieval and downstream QA, with propositions as an alternative unit [67]. - Semantic chunking's extra cost "is not justified by consistent performance gains" over fixed-size chunking [68], so fixed-size baselines must stay in the Module 3 comparison. No chunking study for Tamil was found.

Safety. Madhusudhanan G. [17] studied indirect prompt injection and reasoning monitorability across English, Tamil and Tanglish in a small case study of one model. It is evidence that Tanglish is now studied in LLM safety, a possible later direction for this work.

2.8 Evaluating RAG and LLM output in multilingual and Indic settings

RAG metrics. - RAGAs [69] provides reference-free metrics for faithfulness, answer relevance and context relevance. - ARES [70] fine-tunes lightweight LM judges on synthetic data and corrects their errors with a few hundred human labels through prediction-powered inference. - LLM-as-a-judge can match human preferences, with over 80% agreement for GPT-4, but shows position, verbosity and self-enhancement biases [71].

Multilingual and Indic evaluation. - LLM evaluators are biased towards higher scores, especially in low-resource and non-Latin-script languages, and need calibration against native-speaker judgements [72]. - PARIKSHA [73] ran 90,000 human and 30,000 LLM evaluations across 10 Indic languages and found that human–LLM agreement varies. - In a RAG health chatbot built from real Indian patient queries, factual correctness was lower for Indic-language queries than for English, and code-mixed queries were hard to evaluate [74]. - NoMIRACL [75] shows that many LLMs hallucinate answers when every retrieved passage is non-relevant. - In a 13-language RAG pipeline, Chirkova et al. [76] needed prompt engineering to make the model answer in the user's language and had to adapt metrics to spelling variation in named entities. They list code-switching as an open limitation.

None of these evaluation methods has been validated on Tanglish queries, which is the starting point for Module 4.

2.9 Datasets

The nlpctx/tamil-english-retrieval dataset [18] contains 9,299 LLM-generated query rows over 2,183 Tamil documents from the Mozhi Tamil corpus (IndicCorpV2 news text [47]), with IndicTrans2 [48] English translations. Each row has an English, a Tamil-script, a romanised and a code-mixed query, plus an answer snippet. It is released under Apache-2.0 and has no accompanying paper or reported baselines. To our knowledge, Chapter 5 gives the first retrieval baselines on it.

Among other Tanglish resources: - The questions of Chandu et al. [13] lack a retrieval collection. - TanglishMedBench [57] lacks a collection and relevance judgements. - IndicCMix [23] has parallel renderings but no information needs. - Native-script Tamil retrieval sets [52], [53] can serve as the Tamil-script upper-bound condition.

2.10 Summary of the literature

Work	Year	Languages / script	Task	Method / models	Key result	Gap relative to this project
Lewis et al. [1]	2020	English	Open-domain QA	DPR + BART (RAG)	Retrieval grounding improves QA	English only
Cormack et al. [6]	2009	English	Rank fusion	RRF, k = 60	Beats individual rankers	Not tested across scripts
Bruch et al. [63]	2023	English (BEIR)	Fusion analysis	Convex combination vs RRF	RRF parameter-sensitive; convex better	Same-script only
Chauhan and Agarwal [64]	2026	English (BEIR)	Fusion diagnostics	5 encoders, SPLADE, 3 fusions	Fusion hurts 13–48% of queries	No script mismatch
Feng et al. (LaBSE) [5]	2022	109 languages	Sentence embedding	Dual encoder	Strong bitext retrieval	Native scripts only
Wang et al. (mE5) [3]	2024	~100 languages	Text embedding	Contrastive pre-training	Strong multilingual retrieval	Romanised Tamil untested
Chen et al. (BGE-M3) [4]	2024	100+ languages	Text embedding	Dense + sparse + multi-vector	Strong multilingual retrieval	Romanised Tamil untested
Chari et al. [28]	2025	zh, ru, ar, el, fa (Latinised)	Transliterated-query IR	BGE-M3 etc.; transliterate-train	Large script gap; closed by mixed training	No Indic language
Verma et al. [61]	2026	Many	Representation analysis	LAPE + SAE	Romanised ≠ native ≠ English	Analysis, not retrieval

Work	Year	Languages / script	Task	Method / models	Key result	Gap relative to this project
Gupta et al. [21]	2014	Hindi (Devanagari + Roman)	Mixed-script IR	Autoencoder term equivalents	MRR 0.686 → 0.874	Hindi lyrics; pre-dense, no RAG
FIRE 2013 [24]; MSIR 2015 [20]; MSIR 2013–16 [19]	2013–16	Hindi, Bengali, Gujarati (+ Tamil labelling, 2014–15)	Mixed-script IR	Transliteration, char n-grams, query expansion	Best NDCG@10 0.716 (2015); tweet MAP 0.0377 (2016)	No Tamil retrieval
Ganguly et al. [25]	2014	Hindi (mixed script)	Transliterated search	Rule normalisation + fuzzy queries	~56% over LM baseline	Hindi; pre-dense
Chanda and Pal; Chanda et al. [9], [10]	2024–25	Bengali–English	Code-mixed IR	GPT-3.5, SBERT, BM25, fusion	Best MAP 0.70 (2024); fusion helps (2025)	Bengali; same script; no RAG
Zeng et al. [11]	2026	Chinese / Japanese + English	Code-switching IR	Sparse, dense, late interaction	Drops up to 27%	Not Tamil; not romanised
Madhani et al. (Aksharantar) [22]	2023	21 Indic languages	Transliteration	IndicXlit transformer	26M word pairs	Not evaluated for retrieval
Roark et al. [31]; Kirov et al. [32]	2020; 2024	12 South Asian incl. Tamil	Romanisation data; contextual transliteration	Lexicons + parallel; LM ensembles	Large gains, all 12 languages	Accuracy only, not retrieval
TAMZHI [35]	2024	Romanised Tamil	Back-transliteration	Hybrid rules + ML	Handles dropped vowels	Not evaluated for retrieval
IndicLID [33]; Benton et al. [34]	2023; 2025	22 Indic (native + romanised)	Language ID	Classifiers; synthetic data	Romanised F1 74.7 → 88.2	Sentence-level; no entity handling
Chandu et al. [13]	2018	Hinglish, Tamlish, Tenglish	Factoid QA	Crowd-sourced data	2,848 Tamlish questions	QA, not RAG components
Gupta, Chinnakotla and Shrivastava [36]	2018	Hindi–English	Code-mixed KBQA	Transliterate vs translate	Transliteration +5% / +35%	Hindi; KBQA
Chakravarthi et al. [7], [8]	2020	Tamil–English	Sentiment classification	Corpus + shared task	Tanglish corpus	Classification, not retrieval
Indi-RomCoM [41]; CodeMixQA [40]	2026	Romanised Indic incl. Tamil; 16 CS pairs	LLM instructions; CS QA	Benchmarks	LLMs degrade with mixing	No retrieval
TanglishMedBench [57]	2026	Romanised Tanglish	Medical QA (LLM)	700 QA pairs + expert labels	Benchmark only	No corpus or qrels

Work	Year	Languages / script	Task	Method / models	Key result	Gap relative to this project
Sharma et al. [43]	2026	Hinglish → English	Code-mixed RAG retrieval	Zero-shot vs fine-tuned embedders	96.3% vs 76.7%	Hindi; small task
Gaikwad and Gaikwad [42]	2026	Hindi, Hinglish, English	Code-mixed RAG	Normalisation + abstention layer	Fewer hallucinations	Hindi; custom set
Agarwal [46]	2026	Hinglish (+ Tamil, Telugu probes)	Code-mixed NLU ablation	Normaliser / rules / LLM arbiter	Normaliser redundant after fine-tuning	NLU, not retrieval
Balasubramaniyam and Madushanka [14]	2026	Sinhala, Tamil → English	Cross-lingual IR	Translation vs LaBSE / mE5 / BGE-M3	BGE-M3 Recall@15 95.6% (Tamil)	Native script only
IndicIRSuite [52]; IndicRAGSuite [53]	2024; 2025	11–13 Indic incl. Tamil	IR / RAG benchmarks	Translated MS MARCO; ColBERT	+47.47% MRR@10 (Indic-ColBERT)	Native-script queries only
MIRACL [55]	2023	18 languages	Monolingual retrieval	Human qrels	726k judgements	No Tamil
Singh [15]	2026	English	RAG component study	Parser × chunker × embedder	Components interact	English only
Chhoun et al. [16]; Qu et al. [68]	2026; 2025	Khmer; English	RAG chunking	Chunker comparisons	300-char recursive best; semantic not worth cost	No Tamil
Ma et al. [65]; HyDE [66]	2023	English	Query rewriting for RAG	LLM rewriter; hypothetical docs	Rewriting improves retrieval	Not code-mixed
RAGAs [69]; ARES [70]	2024	English	RAG evaluation	LLM judges	Automated RAG scores	Not validated for Tanglish
Hada et al. [72]; PARIKSHA [73]	2024	8 languages; 10 Indic	LLM-as-judge validity	Human vs LLM judgements	LLM judges inflate scores	No code-mixed queries
Madhusudhanan G. [17]	2026	English, Tamil, Tanglish	LLM safety	Case study, 1 model	Descriptive only	Safety, not retrieval
nlpctx dataset [18]	2026	Tamil, English, romanised, code-mixed	Retrieval data	LLM-generated queries	No baselines published	Synthetic queries; no study
IndicCMix [23]	2026	11 Indic incl. Tamil	Parallel code-mixed text	Translated + romanised renderings	~104.8k Tamil rows	No relevance judgements

Table 2.1: Literature survey summary.

2.11 Research gap

Transliteration-based mixed-script IR has a history at FIRE [19]–[21], [24], [25], and both transliteration plus character-n-gram indexing [20] and rule-based normalisation [25] were already used there. We do not claim either idea as new.

What the surveyed work never combines is: - (a) **Tamil–English code-mixed and romanised queries written by people**; - (b) **retrieval for RAG with modern dense, lexical and fusion components**; - (c) **a controlled comparison of those components, and of query normalisation front-ends, on the same information needs**.

The evidence for each part: - The FIRE retrieval subtasks were Hindi or Bengali [19], [20], [24], [9], [10]. - Every Tamil retrieval and RAG benchmark uses native-script queries [52], [53], [55], [14]. - Code-mixed and transliterated retrieval has been studied for Bengali [9], [10], Hindi [42], [43], Chinese and Japanese [11], and Greek, Arabic, Russian and other non-Indic scripts [28]. - The only Tanglish benchmarks evaluate LLM answers without retrieval [41], [57]. - Transliteration resources report accuracy, not retrieval effect [22], [31], [32], [35]. - RAG components have been compared for English [15], [68] and Khmer [16]. - Fusion has been analysed only where queries and documents share a script [63], [64], [10]. - RAG evaluation methods have not been validated on code-mixed Indian queries [69]–[74].

No prior study evaluates RAG retrieval components (dense, lexical, fusion) and query normalisation for Tamil–English code-mixed and romanised queries. This project fills that intersection. Modules 3–5 extend it to human-written queries, Tamil-aware chunking and generation-side evaluation.

Chapter 3: System Analysis and Design

3.1 Existing system

A typical RAG deployment for an Indian institution today works as follows. Documents are split into fixed-size chunks, embedded with an English or multilingual embedding model, and stored in a vector index. Optionally a BM25 index is added and fused with RRF. The user's query is passed *as typed* to both retrievers.

Drawbacks of the existing approach:

1. **No script handling.** A romanised Tamil query shares no characters with Tamil-script documents, so BM25 cannot match anything, and the embedding model sees an unfamiliar romanised string.
2. **Hybrid fusion assumed safe.** RRF gives the lexical list equal weight even when it matches only incidental tokens, which can push correct dense results down.
3. **Chunkers ignore Tamil.** Character or token chunkers can split Tamil words inside a grapheme cluster, and sentence splitters are tuned for English punctuation.
4. **Evaluated only in English.** Retrieval quality is rarely measured for the query forms real users type.

3.2 Proposed system

The proposed system adds a script-aware front end and a measurement harness to a standard RAG retriever:

1. **Script and language identification per token.** Each Latin-script token is labelled English or romanised Tamil, using English word frequencies (the wordfreq library) plus a list of about 60 common Tanglish particles and function words (*ku, la, enna, pathi, panni, yaar* and so on) that would otherwise be mistaken for English.
2. **Query normalisation.** Romanised Tamil tokens are transliterated into Tamil script; English tokens are kept unchanged. Two transliterators are compared: the ITRANS scheme from the indic-transliteration library, and a Tanglish rule set written for this project.
3. **Multiple retrievers.** BM25 on whole words, BM25 on character 3-grams (robust to spelling variation), and dense retrieval with multilingual-e5-small.
4. **Fusion.** RRF over two lists (BM25 + dense) or three lists (BM25 + character-3-gram BM25 + dense). A BM25 list contains only documents with a positive score.
5. **Evaluation harness.** Recall@k, MRR@10, nDCG@10, bootstrap confidence intervals, paired significance tests and per-stage latency, run for every query form.
6. **Later modules.** A Tamil-aware chunker (Module 3) and an LLM generator with answer-quality evaluation (Module 4).

3.3 Existing versus proposed system

Aspect	Existing RAG pipeline	Proposed system
Query forms	Typed text, treated as one language	English, Tamil script, romanised and code-mixed, each measured separately
Script handling	None	Token-level language ID + transliteration of romanised Tamil

Aspect	Existing RAG pipeline	Proposed system
Lexical matching	Word BM25 only	Word BM25 + character 3-gram BM25
Dense model	Often English-only	Multilingual (mE5-small now; BGE-M3 and LaBSE in Module 3)
Fusion	RRF applied by default	RRF measured per query form; harm detected and reported
Chunking	Fixed size, script-agnostic	Tamil-aware chunker (Module 3)
Evaluation	English test set, often none	2,154 queries × 4 forms, CIs, significance, latency
Hardware	Often GPU	CPU-only (2 cores), reproducible on a laptop

Table 3.1: Existing versus proposed system.

3.4 System architecture

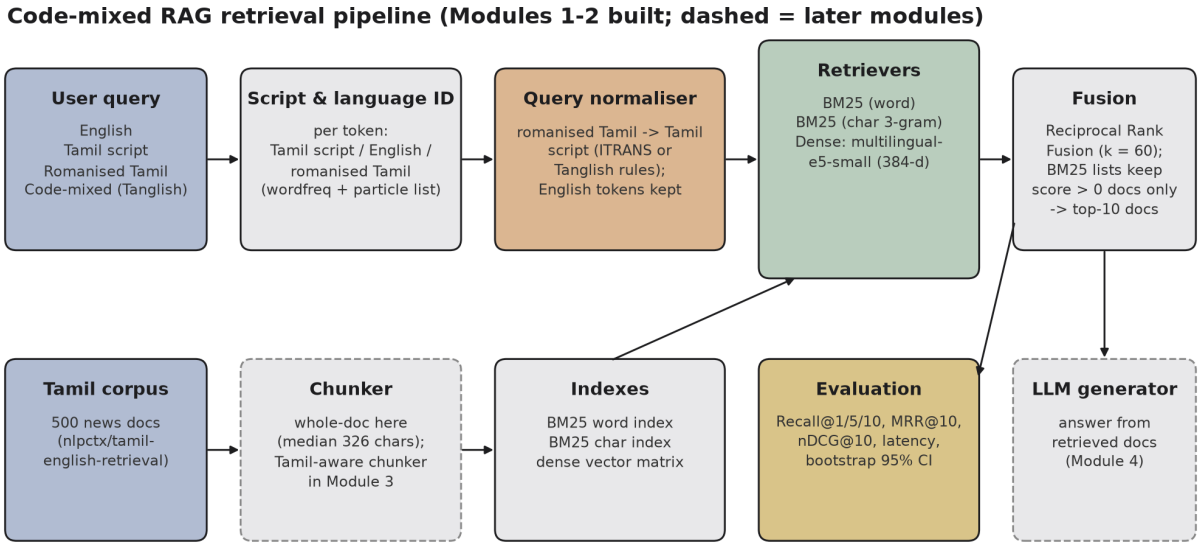


Figure 3.1: Architecture of the proposed code-mixed RAG retrieval pipeline. Solid boxes are built in Modules 1–2; dashed boxes come in later modules.

The query path runs along the top row: the user's query passes through token-level language identification, then the normaliser, then the three retrievers, whose ranked lists are fused by RRF into a top-10 result. The corpus path along the bottom row builds the indexes: documents pass through the chunker (whole documents in Module 2) into a BM25 word index, a BM25 character index and a dense vector matrix. The evaluation block compares the top-10 list with the gold document. The LLM generator is added in Module 4.

3.5 Module description

Module	Content	Status
M1: Problem and design	Problem statement, objectives, literature survey, existing vs proposed system, architecture, methodology	Complete (this report)
M2: Data and baseline retrieval	Dataset subset, query audit, normaliser, 5 retrievers, metrics, results, examples	Complete (this report)
M3: Human Tanglish set and chunking	~500 human-written Tanglish questions; longer documents; Tamil-aware chunker vs ChunkLab chunkers; BGE-M3 and LaBSE	Planned
M4: Generation-side evaluation	LLM generator; answer correctness and faithfulness per query form	Planned

Module	Content	Status
M5: Paper and release	Paper, public benchmark and code	Planned

Table 3.2: Project modules.

3.6 Hardware and software

Hardware used for every Module 2 experiment: 2 virtual CPU cores (ARM64), 7 GB RAM, no GPU.

Software: Python 3.12; PyTorch 2.15.0 (CPU build); sentence-transformers 6.2.0; transformers 5.22.1; rank-bm25 0.2.2; indic-transliteration 2.3.82; wordfreq 3.1.1; pandas 3.1; NumPy 2.5; matplotlib 3.11; huggingface_hub 2.4. The ChunkLab code base [12] supplied the retriever and metric design (BM25, dense, hybrid RRF with k = 60, and the Precision/Recall/MRR/nDCG definitions). Module 2 reimplements these as small standalone functions so the experiment runs without ChunkLab's Streamlit interface.

Chapter 4: Methodology

4.1 Research questions

- **RQ1.** How much does retrieval quality drop when the same information need is expressed in romanised or code-mixed Tamil instead of Tamil script or English?
- **RQ2.** Which retriever type (lexical word, lexical character n-gram, dense, hybrid) is most robust to each query form?
- **RQ3.** How much of the drop does transliteration-based query normalisation recover, at what latency cost, and does a Tanglish-specific rule set beat a generic scheme?
- **RQ4 (Module 3).** Does Tamil-aware chunking improve retrieval on longer documents for each query form?
- **RQ5 (Module 4).** How do retrieval differences carry through to generated-answer quality?

Module 2 answers RQ1 to RQ3.

4.2 Dataset preparation

The source is nlpctx/tamil-english-retrieval [18] (9,299 query rows over 2,183 documents, all news). To keep the study CPU-feasible, 500 documents were sampled uniformly at random with a fixed seed (42), and **every** query row generated for those documents was kept, giving 2,154 queries. Each query has exactly one relevant document, the one it was generated from. Each query exists in all four forms, so every form is evaluated on the same 2,154 information needs, which allows paired comparisons.

Two corpora were built from the same 500 documents: the **Tamil corpus** (original Tamil-script text; the main setting) and the **English corpus** (the dataset's IndicTrans2 English translation; a secondary setting). No chunking was applied: documents have a median length of 326 characters, so each fits easily in the embedding model's 512-token limit.

4.3 Query normalisation

Normalisation applies to romanised and code-mixed queries. Each query is split into runs of Latin letters and other characters. A Latin token is kept as English if its Zipf frequency in English (from wordfreq) is at least 3.0 and it is not in the Tanglish particle list; otherwise it is treated as romanised Tamil and transliterated. Digits and punctuation pass through unchanged.

ITRANS mode applies the ITRANS-to-Tamil mapping of the indic-transliteration library to the lower-cased token. ITRANS is a formal romanisation scheme designed for Sanskrit-family text. For example, it maps *e* to the long vowel ஏ and *n* to the dental ந in every position, which does not match how Tanglish is typed.

Tanglish rules mode is a greedy longest-match transliterator written for this project. Its complete rule set:

Tanglish spelling	Tamil output	Example (actual output of the code)
zh	ழ	kizhakku → கிழக்கு

Tanglish spelling	Tamil output	Example (actual output of the code)
th, dh	த	thadai → தடை
t, d (not followed by h)	ட	tamil → டமில்
tr, rr	ற்ற	payitrapatta → பயிற்றபட்ட
ndr	ன்ற	endru → என்று
nth, ndh	ந்த	vandhu → வந்து
nd, nt	ண்ட (ந்த at the start of a word)	kandu → கண்டு
ng, ngk	ங்க	ingu → இங்கு
nj, nch	ஞ்ச	manjal → மஞ்சல்
ny	ஞ	nyayam → ஞயம்
n at the start of a word	ந	neelam → நீலம்
other n; nn	ன; ன்ன	enna → என்ன
ch, s	ச	seyarkai → செயர்கை
tch	ட்ச	kaatchi → காட்சி
sh; ksh	ஷ; க்ஷ	shankar → ஷன்கர்
k, g, c, q, kh, gh	க	gangai → கங்கை
p, b, f, ph, bh	ப	bharatham → பரதம்
j, z; h; x	ஜ; ஹ; க்ஸ	haridwar → ஹரிட்வர்
m, y, r, l; v, w	ம, ய, ர, ல; வ	vaazhai → வாழை
doubled consonant other than r, n (kk, pp, tt, ll…)	consonant + pulli + consonant	kudikkalama → குடிக்கலம
aa; ee or ii; oo or uu; ai; au; ae	vowel signs, shown on க: கா; கீ; கூ; கை; கௌ; கே	maagaana → மாகான
a, i, u, e, o after a consonant	shown on க: க (inherent a); கி; கு; கெ; கொ	kurithu → குரிது
consonant with no following vowel	consonant + pulli (்)	yaar → யார்
vowel at the start of a word	independent vowel (அ, ஆ, இ…)	aasiriyar → ஆசிரியர்

Table 4.1: The complete rule set of the Tanglish-to-Tamil transliterator, as implemented in normalise.py (longest match first). Every example is the code's actual output.

The rules are deliberately simple and were written before the evaluation was run, from knowledge of Tamil orthography only; they were not tuned on the test queries. They cannot recover distinctions that Tanglish spelling does not encode (ல/ள typed as *l*, ன/ண/ந as *n*, or த/ட both typed as *t*: *tamil* becomes டமில் not தமிழ், which is why character n-gram matching is also evaluated.

4.4 Retrievers

1. **BM25 (word).** Okapi BM25 with rank-bm25 defaults (k1 = 1.5, b = 0.75). Tokens are maximal runs of Latin letters, digits and Tamil-block characters (U+0B80 to U+0BFF),

lower-cased. Including the whole Tamil block keeps vowel signs attached to their consonants.

2. **BM25 (character 3-gram).** The same BM25 over character 3-grams of each word with boundary markers (#word#). It tolerates spelling variation, which matters for transliteration output.
3. **Dense (multilingual-e5-small).** 118M parameters, 384-dimensional embeddings, with the required "query:" and "passage:" prefixes. Vectors are L2-normalised and ranked by dot product, which equals cosine similarity. This small model was chosen so every experiment runs on a 2-core CPU.
4. **Hybrid-RRF.** Reciprocal Rank Fusion of the top-100 lists from BM25 (word) and dense, with k = 60, as in ChunkLab. rank-bm25 gives every document that shares no term with the query a score of exactly 0; such documents are removed from the BM25 list before fusion, so a query with no lexical match contributes no BM25 list at all.
5. **Hybrid-RRF3.** RRF of BM25 (word), BM25 (character 3-gram) and dense, with the same zero-score filter.

Correction to an earlier run. The first version of this pipeline (and an earlier draft of this report) passed the full top-100 BM25 list to RRF. When a query matched nothing, all scores were 0 and the "ranking" was simply documents 0 to 99 in index order, which RRF then rewarded. This affected 72% of English and 77% of code-mixed queries on the Tamil corpus (no word-BM25 match). Section 5.6 reports the corrected numbers and keeps the old ones, labelled *naive*, for comparison. The single BM25 retrievers are also filtered now, which removes a few chance hits among zero-score documents and lowers some BM25 scores by up to 0.022 Recall@10.

4.5 Evaluation metrics

Each query q has one relevant document; let r(q) be its rank in the top-10 list (undefined if absent). Over the query set:

- **Recall@k** (k = 1, 5, 10): the fraction of queries whose relevant document appears in the top k. With one relevant document per query this equals the hit rate.
- **MRR@10** (Mean Reciprocal Rank): the average of 1 / r(q), counting 0 when the relevant document is not in the top 10.
- **nDCG@10**: the average of 1 / log2(r(q) + 1), counting 0 when absent. With one relevant document the ideal DCG is 1, so no further normalisation is needed.
- **Latency**: wall-clock time per query for each stage (normalisation, BM25 scoring, dense query encoding, dense search, fusion), measured one query at a time on 200 random queries; median and 95th percentile are reported.

Statistical testing. 95% confidence intervals for MRR@10 use 1,000 bootstrap resamples of queries. Differences between two configurations on the same queries use a two-sided paired bootstrap test with 2,000 resamples; with 2,000 resamples, the smallest reportable p-value is below 0.001. Because queries cluster within documents (2,154 queries from 500 documents, about 4.3 per document), every key comparison is repeated with a stricter *cluster* bootstrap that resamples whole documents together with all their queries.

4.6 Experimental protocol

1. Sample 500 documents (seed 42) and keep all 2,154 associated queries.
2. Create the ITRANS and rules versions of each romanised and code-mixed query, giving eight query sets: English, Tamil script, romanised, code-mixed, romanised + ITRANS, romanised + rules, code-mixed + ITRANS, code-mixed + rules.
3. Encode both corpora and all eight query sets with multilingual-e5-small; build the word and character BM25 indexes.
4. Run all five retrievers for all eight query sets on the Tamil corpus (40 runs) and for the four raw query sets on the English corpus (20 runs): 60 main runs. The two naive fusion variants (Section 4.4) are re-run on all 12 query sets for comparison, giving 84 runs and 180,936 ranked lists.
5. Compute metrics, confidence intervals, query-level and document-cluster significance tests and latency; audit query composition; extract example outputs.

Chapter 5: Module 2 Implementation and Results

5.1 Dataset statistics

Property	Value
Source	nlpctx/tamil-english-retrieval (Hugging Face, Apache-2.0)
Full dataset	2,183 documents, 9,299 query rows, all news domain
Documents sampled (seed 42)	500
Queries per form	2,154 (all rows for the sampled documents)
Query forms	4 raw (English, Tamil script, romanised, code-mixed) + 4 normalised = 8 query sets
Relevant documents per query	1 (the source document)
Document length (Tamil, characters)	mean 391, median 326, max 2,795
Corpora	Tamil script (main); English translation (secondary)
Ranked lists evaluated	84 runs (60 main + 24 naive-fusion comparison runs) × 2,154 queries = 180,936

Table 5.1: Evaluation subset.

5.2 Query audit: how code-mixed are the queries?

Before looking at retrieval scores, we examined the queries themselves.

Query set	Avg tokens	Latin-script tokens (%)	Latin tokens judged English (%)	Tamil-script tokens found in corpus vocabulary (%)
English	5.6	98.9	85.1	n/a
Tamil script	5.0	0.3	n/a	77.7
Romanised	5.1	98.9	14.2	0.0
Code-mixed	5.7	98.9	69.0	n/a
Rom -> ITRANS	5.1	14.0	100.0	15.2
Rom -> rules	5.1	13.8	100.0	29.9
CM -> ITRANS	5.7	68.2	100.0	16.6
CM -> rules	5.7	68.2	100.0	24.9

Table 5.2: Token composition of each query set (2,154 queries each). After normalisation, every remaining Latin token is one the language-ID step judged English, so that column is 100% by construction.

Audit measure	Value
Code-mixed queries identical to the English query	13.3%
Code-mixed queries with no romanised-Tamil token	22.0%
Mean romanised-Tamil tokens per code-mixed query	1.74
Mean romanised-Tamil tokens per romanised query	4.30

Table 5.3: How much Tamil the dataset's code-mixed queries contain.

The audit gives two findings that shape how the results should be read:

1. **The romanised queries are genuinely Tamil.** They contain on average 4.3 tokens detected as romanised Tamil, and only 14% of their Latin tokens look English, mostly names such as *Colombia* or *London*.
2. **The "code-mixed" queries are mostly English.** On average only 1.74 tokens per code-mixed query are Tamil; 22.0% contain no Tamil token at all, and 13.3% are identical to the English query. This is a property of the LLM that generated the dataset, not of how people type Tanglish. Results for the code-mixed set therefore mostly measure English-to-Tamil cross-lingual retrieval, which is the main reason Module 3 will collect human-written Tanglish questions.

Transliteration also changes vocabulary overlap with the corpus. Of the Tamil-script tokens in native Tamil queries, 77.7% appear in the corpus vocabulary. After rule-based transliteration of romanised queries, 29.9% do, against 15.2% for ITRANS. Exact word matches remain rare even after normalisation, which is why character n-gram BM25 benefits most.

5.3 Main results: raw queries on the Tamil corpus

Query form	Retriever	R@1	R@5	R@10	MRR@10	MRR@10 95% CI	nDCG@10
Tamil script	BM25	0.764	0.872	0.891	0.811	[0.795, 0.825]	0.830
	BM25-char3	0.898	0.954	0.964	**0.923**	[0.912, 0.933]	0.933
	Dense-e5	0.862	0.956	0.974	0.903	[0.893, 0.913]	0.921
	Hybrid-RRF	0.860	0.937	0.960	0.895	[0.884, 0.907]	0.911
	Hybrid-RRF3	0.883	0.945	0.970	0.911	[0.900, 0.921]	0.925
English	BM25	0.062	0.075	0.078	0.067	[0.057, 0.078]	0.070
	BM25-char3	0.051	0.075	0.088	0.062	[0.053, 0.072]	0.068
	Dense-e5	0.711	0.892	0.937	**0.788**	[0.774, 0.803]	0.824
	Hybrid-RRF	0.567	0.889	0.936	0.710	[0.695, 0.726]	0.766
	Hybrid-RRF3	0.085	0.695	0.879	0.305	[0.295, 0.316]	0.442
Code-mixed	BM25	0.065	0.075	0.079	0.069	[0.059, 0.080]	0.072
	BM25-char3	0.052	0.078	0.088	0.063	[0.054, 0.073]	0.069
	Dense-e5	0.597	0.805	0.862	**0.687**	[0.670, 0.706]	0.729
	Hybrid-RRF	0.505	0.806	0.863	0.637	[0.622, 0.655]	0.693
	Hybrid-RRF3	0.085	0.616	0.801	0.278	[0.268, 0.290]	0.403
Romanised	BM25	0.051	0.063	0.068	0.056	[0.046, 0.065]	0.059
	BM25-char3	0.048	0.068	0.082	0.058	[0.048, 0.067]	0.064
	Dense-e5	0.093	0.212	0.294	0.143	[0.132, 0.157]	0.178
	Hybrid-RRF	0.129	0.247	0.321	**0.178**	[0.164, 0.192]	0.211
	Hybrid-RRF3	0.065	0.178	0.262	0.113	[0.103, 0.123]	0.148

Table 5.4: Raw queries on the Tamil corpus (n = 2,154 per form). Bold marks the best MRR@10 per form.

RQ1 and RQ2. With Tamil-script queries every retriever works well (MRR@10 between 0.811 and 0.923). English queries are retrieved well only by the dense model (0.788), because BM25 cannot match English words to Tamil text. Code-mixed queries behave like English queries (dense 0.687). **Romanised queries fail everywhere:** dense retrieval

reaches MRR@10 = 0.143 and Recall@10 = 0.294, so for 7 in 10 romanised questions the correct document is not in the top 10; the best raw configuration (BM25 + dense fusion) reaches only 0.178. The multilingual embedding model, although trained on about 100 languages, does not place romanised Tamil near Tamil script.

Why character 3-grams beat word BM25 on native Tamil (0.923 against 0.811). Tamil is agglutinative: case markers, postpositions and clitics attach to the stem, so one noun appears in many surface forms (மோதல்மோதலுக்கு, மோதலின். No Tamil stemmer or lemmatiser was used, so word BM25 counts these as unrelated terms, and a query word matches a document only if the suffix is identical. Character 3-grams share the stem's grams across all these forms, which acts as an approximate, language-independent stemmer. Only 77.7% of native-query Tamil tokens occur verbatim in the corpus vocabulary (Section 5.2), which is the gap the 3-grams close.

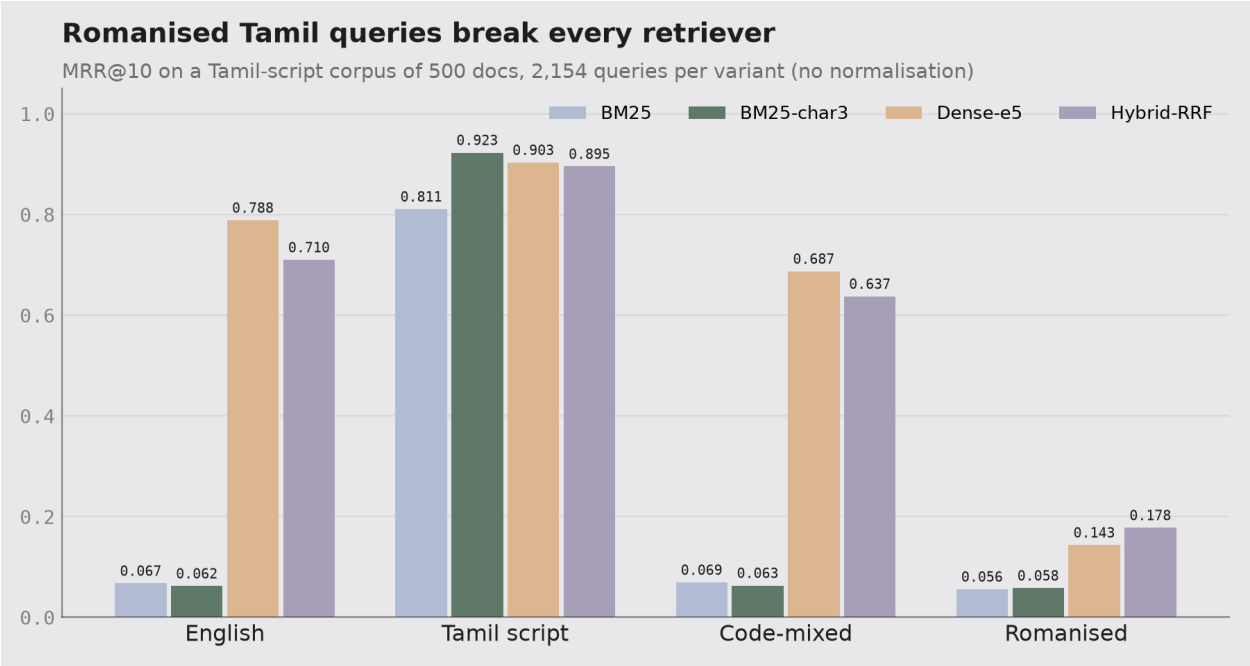


Figure 5.1: MRR@10 for each query form and retriever on the Tamil corpus, without normalisation.

5.4 Effect of query normalisation

Query form	Retriever	R@1	R@5	R@10	MRR@10	MRR@10 95% CI	nDCG@10
Romanised	BM25	0.051	0.063	0.068	0.056	[0.046, 0.065]	0.059
	BM25-char3	0.048	0.068	0.082	0.058	[0.048, 0.067]	0.064
	Dense-e5	0.093	0.212	0.294	0.143	[0.132, 0.157]	0.178
	Hybrid-RRF	0.129	0.247	0.321	**0.178**	[0.164, 0.192]	0.211
	Hybrid-RRF3	0.065	0.178	0.262	0.113	[0.103, 0.123]	0.148
Romanised + ITRANS	BM25	0.174	0.257	0.290	0.210	[0.194, 0.226]	0.229
	BM25-char3	0.475	0.638	0.705	**0.547**	[0.529, 0.567]	0.585
	Dense-e5	0.253	0.434	0.518	0.328	[0.310, 0.346]	0.373
	Hybrid-RRF	0.305	0.487	0.592	0.386	[0.366, 0.404]	0.435
	Hybrid-RRF3	0.417	0.631	0.718	0.506	[0.486, 0.522]	0.556
Romanised + rules	BM25	0.279	0.405	0.449	0.333	[0.314, 0.351]	0.361
	BM25-char3	0.623	0.766	0.812	**0.684**	[0.666, 0.702]	0.715

Query form	Retriever	R@1	R@5	R@10	MRR@10	MRR@10 95% CI	nDCG@10
	Dense-e5	0.443	0.657	0.746	0.535	[0.515, 0.554]	0.585
	Hybrid-RRF	0.448	0.648	0.742	0.537	[0.518, 0.556]	0.586
	Hybrid-RRF3	0.539	0.743	0.842	0.627	[0.609, 0.646]	0.678
Code-mixed	BM25	0.065	0.075	0.079	0.069	[0.059, 0.080]	0.072
	BM25-char3	0.052	0.078	0.088	0.063	[0.054, 0.073]	0.069
	Dense-e5	0.597	0.805	0.862	**0.687**	[0.670, 0.706]	0.729
	Hybrid-RRF	0.505	0.806	0.863	0.637	[0.622, 0.655]	0.693
	Hybrid-RRF3	0.085	0.616	0.801	0.278	[0.268, 0.290]	0.403
Code-mixed + ITRANS	BM25	0.080	0.109	0.116	0.093	[0.082, 0.105]	0.099
	BM25-char3	0.145	0.267	0.322	0.197	[0.184, 0.213]	0.227
	Dense-e5	0.570	0.793	0.859	**0.666**	[0.649, 0.685]	0.713
	Hybrid-RRF	0.431	0.777	0.855	0.585	[0.569, 0.604]	0.651
	Hybrid-RRF3	0.294	0.584	0.703	0.412	[0.395, 0.431]	0.481
Code-mixed + rules	BM25	0.097	0.128	0.135	0.111	[0.098, 0.124]	0.117
	BM25-char3	0.181	0.312	0.369	0.238	[0.223, 0.254]	0.269
	Dense-e5	0.597	0.816	0.880	**0.692**	[0.676, 0.710]	0.738
	Hybrid-RRF	0.422	0.745	0.841	0.568	[0.552, 0.586]	0.634
	Hybrid-RRF3	0.304	0.590	0.704	0.420	[0.402, 0.438]	0.488

Table 5.5: Effect of query normalisation on the Tamil corpus. Bold marks the best MRR@10 per query set.

RQ3. For romanised queries, normalisation is the largest effect in the study:

- Rules + character-3-gram BM25 raises MRR@10 from 0.058 to **0.684** and Recall@10 from 0.082 to **0.812**.
- Rules + dense raises MRR@10 from 0.143 to 0.535.
- Rules + RRF3 gives the best Recall@10 of all (0.842), useful when the generator reads the top 10, although its MRR@10 (0.627) is below character-3-gram BM25 alone (0.684).
- The Tanglish rules beat ITRANS on every retriever (character-3-gram BM25: 0.684 against 0.547 MRR@10).
- A gap remains: the best romanised result (0.684) is still well below Tamil script (0.923), because Tanglish spelling loses distinctions such as ல/ள, ன/ண/ந and த/ட.

For code-mixed queries, normalisation helps the lexical retrievers (BM25 0.069 to 0.111; RRF3 0.278 to 0.420) but **does not change dense retrieval** (0.687 to 0.692, p = 0.45). These queries are mostly English (Section 5.2), so there is little Tamil to transliterate, and the dense model already handles the English part.

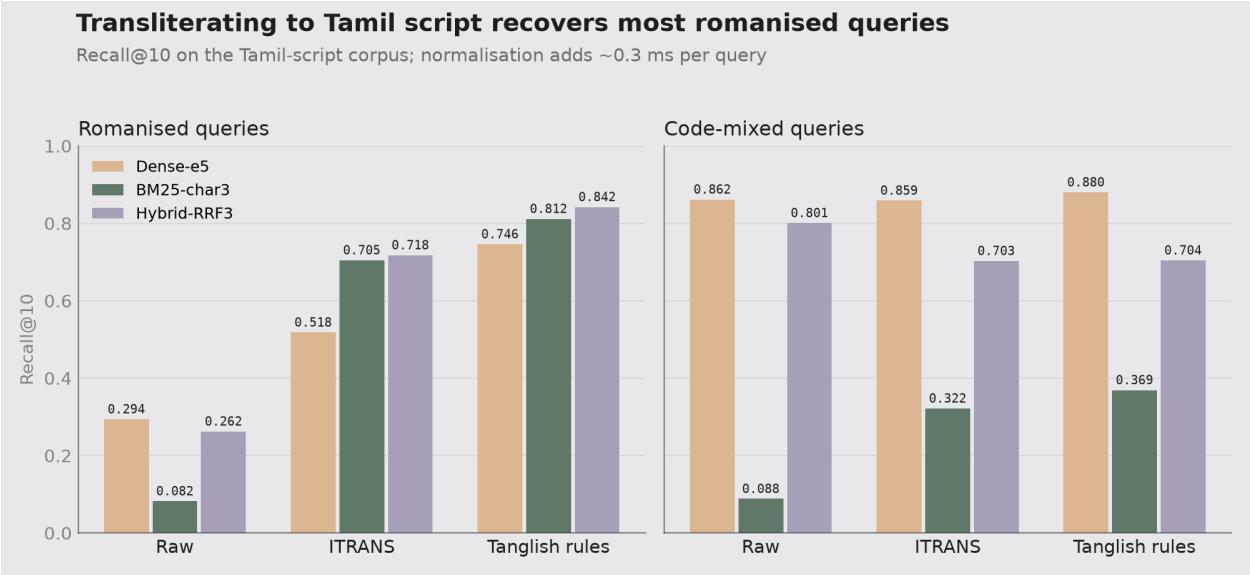


Figure 5.2: Recall@10 for raw, ITRANS-normalised and rule-normalised romanised and code-mixed queries.

5.5 Statistical significance

Comparison	Retriever(s)	MRR@10 before	MRR@10 after	Change	p (query bootstrap)	p (cluster bootstrap)	Change, 95% CI (cluster)
Romanised → Romanised + rules	Dense	0.143	0.535	+0.391	< 0.001	< 0.001	[+0.365, +0.417]
Romanised → Romanised + rules	BM25 char-3	0.058	0.684	+0.626	< 0.001	< 0.001	[+0.596, +0.655]
Romanised → Romanised + rules	Hybrid-RRF3	0.113	0.627	+0.514	< 0.001	< 0.001	[+0.485, +0.542]
Romanised + ITRANS → Romanised + rules	Dense	0.328	0.535	+0.207	< 0.001	< 0.001	[+0.184, +0.230]
Romanised + ITRANS → Romanised + rules	BM25 char-3	0.547	0.684	+0.137	< 0.001	< 0.001	[+0.113, +0.161]
Code-mixed → Code-mixed + rules	Dense	0.687	0.692	+0.005	0.446	0.573	[-0.013, +0.022]
Code-mixed → Code-mixed + rules	BM25	0.069	0.111	+0.042	< 0.001	< 0.001	[+0.024, +0.062]
Code-mixed → Code-mixed + rules	Hybrid-RRF3	0.278	0.420	+0.142	< 0.001	< 0.001	[+0.115, +0.167]
Tamil script: fusion	Dense → Hybrid-RRF	0.903	0.895	-0.008	0.168	0.287	[-0.023, +0.007]
Tamil script: fusion	Dense → Hybrid-RRF (naive, v1)	0.903	0.884	-0.019	0.003	0.032	[-0.036, -0.002]
English: fusion	Dense → Hybrid-RRF	0.788	0.710	-0.079	< 0.001	< 0.001	[-0.090, -0.068]
English: fusion	Dense → Hybrid-RRF (naive, v1)	0.788	0.187	-0.601	< 0.001	< 0.001	[-0.634, -0.567]
Code-mixed: fusion	Dense → Hybrid-RRF	0.687	0.637	-0.050	< 0.001	< 0.001	[-0.062, -0.038]
Code-mixed: fusion	Dense → Hybrid-RRF (naive, v1)	0.687	0.181	-0.506	< 0.001	< 0.001	[-0.540, -0.471]

Comparison	Retriever(s)	MRR@10 before	MRR@10 after	Change	p (query bootstrap)	p (cluster bootstrap)	Change, 95% CI (cluster)
Romanised: fusion	Dense → Hybrid-RRF	0.143	0.178	+0.034	< 0.001	< 0.001	[+0.023, +0.046]
Romanised: fusion	Dense → Hybrid-RRF (naive, v1)	0.143	0.092	-0.052	< 0.001	< 0.001	[-0.075, -0.030]
Romanised + rules: fusion	Dense → Hybrid-RRF	0.535	0.537	+0.002	0.803	0.856	[-0.023, +0.026]
Romanised + rules: fusion	Dense → Hybrid-RRF (naive, v1)	0.535	0.441	-0.094	< 0.001	< 0.001	[-0.126, -0.062]
Romanised + rules: fusion	BM25 char-3 → Hybrid-RRF3	0.684	0.627	-0.057	< 0.001	< 0.001	[-0.080, -0.034]

Table 5.6: Paired significance tests on the Tamil corpus (2,000 resamples each). The query bootstrap resamples queries; the cluster bootstrap resamples the 500 source documents with all their queries. Changes are computed from unrounded values, so a change can differ by 0.001 from the difference of the rounded MRR@10 columns. "Naive, v1" is the earlier fusion that included zero-score BM25 documents.

Every improvement from the Tanglish rules for romanised queries is significant ($p < 0.001$), as is the rules-over-ITRANS improvement, under both the query-level and the stricter document-cluster bootstrap. The dense change for code-mixed queries is not significant ($p = 0.45$; cluster $p = 0.57$). The cross-script losses caused by equal-weight fusion remain significant under both tests; the small loss for Tamil-script queries does not ($p = 0.17$; cluster $p = 0.29$). The cluster bootstrap widens the MRR@10 confidence intervals by about 1.6 times (for example dense on English queries: query-level [0.774, 0.803], cluster [0.766, 0.810]), but no conclusion in this report changes.

5.6 Hybrid fusion can hurt

Hybrid BM25 + dense retrieval with RRF is widely recommended as a robust default. With equal weights and no script check, it is not robust across scripts. When the query and the documents are in different scripts (English or code-mixed queries against Tamil text), word BM25 matches nothing for most queries, and for the rest it matches only incidental tokens: the commonest English words that also occur in the Tamil documents are *of*, *tamil*, *sri*, *from*, *news* and *a*. RRF still gives that list the same weight as the dense list, so the correct document ranked high by the dense model is pushed down. With zero-score documents correctly excluded:

- English queries: dense 0.788, Hybrid-RRF 0.710 (−0.079, $p < 0.001$; cluster $p < 0.001$).
- Code-mixed queries: dense 0.687, Hybrid-RRF 0.637 (−0.050, $p < 0.001$; cluster $p < 0.001$).
- Adding the character-3-gram list makes it much worse (English 0.305, code-mixed 0.278), because character 3-grams of English words also match grams of the digits and stray Latin words inside Tamil documents (median 10 matching documents per English query), and these weak matches displace the dense result.
- For Tamil-script queries, fusion is slightly below dense alone (0.903 to 0.895, not significant) and below character-3-gram BM25 alone (0.923).

The bug and what it changed. The earlier, naive run fed zero-score documents into RRF in index order, which turned "BM25 found nothing" into "BM25 strongly recommends documents 0 to 99". That inflated the harm seven- to tenfold: English 0.788 → 0.187 (−0.601) and code-mixed 0.687 → 0.181 (−0.506). The corrected losses are much smaller, but they are

still significant, so the conclusion that equal-weight hybrid fusion hurts across scripts stands, at its corrected size.

Fusion helps when the lexical list carries real signal. For raw romanised queries the few word-BM25 matches are mostly names and numbers (*venugopal*, *amir*, *2000*, *1984*), and fusion lifts MRR@10 from 0.143 to 0.178 ($p < 0.001$). After rule normalisation, RRF3 gives the best Recall@10 (0.842). **Practical implication:** a code-mixed RAG system should choose or weight retrievers according to the query's script, and must drop zero-score lexical results before fusing, rather than fuse blindly.

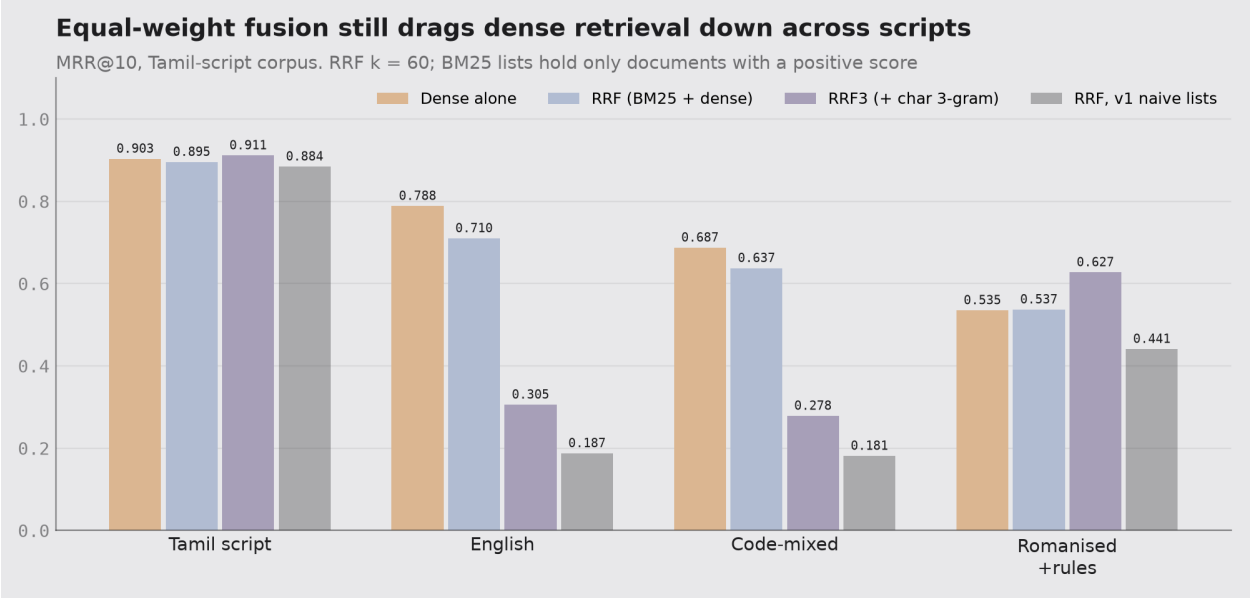


Figure 5.3: MRR@10 of dense retrieval alone against the corrected RRF and RRF3 hybrids, with the earlier naive RRF (zero-score documents not removed) for contrast.

5.7 Secondary setting: English corpus

Query form	Retriever	R@1	R@5	R@10	MRR@10	MRR@10 95% CI	nDCG@10
English	BM25	0.838	0.918	0.929	0.870	[0.857, 0.883]	0.885
	BM25-char3	0.845	0.914	0.937	0.876	[0.863, 0.888]	0.891
	Dense-e5	0.915	0.972	0.984	**0.939**	[0.929, 0.947]	0.950
	Hybrid-RRF	0.894	0.950	0.964	0.920	[0.909, 0.930]	0.930
	Hybrid-RRF3	0.890	0.950	0.964	0.916	[0.905, 0.926]	0.927
Tamil script	BM25	0.058	0.071	0.076	0.064	[0.054, 0.074]	0.067
	BM25-char3	0.036	0.067	0.084	0.050	[0.041, 0.059]	0.058
	Dense-e5	0.664	0.850	0.896	**0.742**	[0.726, 0.760]	0.779
	Hybrid-RRF	0.643	0.846	0.896	0.728	[0.713, 0.746]	0.769
	Hybrid-RRF3	0.089	0.132	0.212	0.113	[0.101, 0.127]	0.135
Code-mixed	BM25	0.833	0.905	0.922	0.865	[0.851, 0.878]	0.879
	BM25-char3	0.820	0.899	0.916	0.855	[0.840, 0.868]	0.870
	Dense-e5	0.838	0.923	0.942	0.875	[0.861, 0.888]	0.891
	Hybrid-RRF	0.870	0.938	0.957	0.900	[0.888, 0.910]	0.914
	Hybrid-RRF3	0.875	0.935	0.952	0.901	[0.889, 0.912]	0.913
Romanised	BM25	0.493	0.560	0.571	0.522	[0.503, 0.541]	0.534

Query form	Retriever	R@1	R@5	R@10	MRR@10	MRR@10 95% CI	nDCG@10
	BM25-char3	0.392	0.518	0.560	0.445	[0.426, 0.465]	0.472
	Dense-e5	0.278	0.412	0.466	0.333	[0.314, 0.353]	0.364
	Hybrid-RRF	0.506	0.595	0.619	0.544	[0.524, 0.564]	0.562
	Hybrid-RRF3	0.520	0.605	0.629	**0.557**	[0.537, 0.577]	0.575

Table 5.7: Raw queries on the English-translation corpus. Bold marks the best MRR@10 per form.

When the documents are English translations, English and code-mixed queries are easy (dense 0.939; RRF3 0.901 for code-mixed). Tamil-script queries are handled only by the dense model (0.742; word-BM25 + dense fusion 0.728). Romanised queries are again the hardest form (best 0.557 with RRF3); lexical retrievers do comparatively well on them because romanised queries keep English named entities. Whatever the document language, romanised Tamil is the weakest query form.

5.8 Latency

Stage	Median (ms)	95th percentile (ms)
Tanglish rule normalisation	0.34	0.48
BM25 (word) scoring + top-k	0.67	0.98
BM25 (char 3-gram) scoring + top-k	4.88	8.59
Dense query encoding (mE5-small, CPU)	32.11	39.26
Dense search (500 × 384 dot product)	0.14	0.18
RRF fusion of 3 lists	0.19	0.23
Full rules + RRF3 pipeline (sum of medians)	**38.3**	

Table 5.8: Per-query latency, one query at a time on 2 CPU cores (200 random romanised queries; index build and document encoding excluded).

The normaliser costs about 0.3 ms per query, which is negligible. Dense query encoding dominates (about 32 ms on 2 CPU cores). The best romanised pipeline (rules + character-3-gram BM25) needs about 5 ms per query with no neural model at all, and the full rules + RRF3 pipeline about 38 ms (sum of stage medians). All are well within interactive limits.

5.9 Example outputs

The following are real query-to-retrieval outputs of the dense retriever on the Tamil corpus. “Rank of gold” is the position of the correct document in the top 10; a dash means it is not in the top 10.

Example 1. Gold document (English translation, shortened): “A guerrilla attack on military personnel in the municipality of Aragita, in the Arawka region of Colombia, has killed 5 soldiers.”

Form	Query as given to the retriever	Rank of gold	Top-1 retrieved document (Tamil, shortened)
English	Colombia military soldiers killed in attack	1	✓ correct: கொலம்பியா நாட்டின் அராவ்கா துறைக்கு உட்பட்ட அராகிட்டா நகராட்சி பகுதியி…

Form	Query as given to the retriever	Rank of gold	Top-1 retrieved document (Tamil, shortened)
Tamil script	தாக்குதலில் கொலம்பியா ராணுவ வீரர்கள் உயிரிழப்பு	1	✓ correct: கொலம்பியா நாட்டின் அராவ்கா துறைக்கு உட்பட்ட அராகிட்டா நகராட்சி பகுதியி…
Roman-ised	thakkudhalil Colombia raanuva thiraigal uyirizhappu	—	✗ wrong: பஞ்சாலை தொழிலாளர்களின் வாழ்வியலையும்உடுமலை வட்டாரப் பகுதிகளில் முன்ப…
Roman-ised + rules	தக்குதலில் Colombia ரானுவ திரைகல் உயிரிழப்பு	1	✓ correct: கொலம்பியா நாட்டின் அராவ்கா துறைக்கு உட்பட்ட அராகிட்டா நகராட்சி பகுதியி…
Code-mixed	Colombia military soldiers killed in attack	1	✓ correct: கொலம்பியா நாட்டின் அராவ்கா துறைக்கு உட்பட்ட அராகிட்டா நகராட்சி பகுதியி…
Code-mixed + rules	Colombia military soldiers killed in attack	1	✓ correct: கொலம்பியா நாட்டின் அராவ்கா துறைக்கு உட்பட்ட அராகிட்டா நகராட்சி பகுதியி…

Example 2. Gold document (English translation, shortened): “A member of the Northern Province ஜி.ரி.லிங்கநாதன்has demanded that the length of the uniform of the schoolgirls of the Northern Province b…”

Form	Query as given to the retriever	Rank of gold	Top-1 retrieved document (Tamil, shortened)
English	Northern Province school uniform length request	1	✓ correct: வடக்கு மாகாண பாடசாலை மாணவிகளின் சீருடையின் நீளத்தை அதிகரிக்க வேண்டும்..
Tamil script	வடக்கு மாகாண பாடசாலை சீருடை நீளம் கோரிக்கை	1	✓ correct: வடக்கு மாகாண பாடசாலை மாணவிகளின் சீருடையின் நீளத்தை அதிகரிக்க வேண்டும்..
Roman-ised	Vadakku maagaana paadasaalai seerudai neelam korikkai	—	✗ wrong: “பின்னே என்ன சார்? ஒருத்தரை வெறுக்கிறோம்னா அதை வெளிப்படையா சொல்லலாமே? …
Roman-ised + rules	வடக்கு மாகான பாடசாலை சீருடை நீலம் கொரிக்கை	1	✓ correct: வடக்கு மாகாண பாடசாலை மாணவிகளின் சீருடையின் நீளத்தை அதிகரிக்க வேண்டும்..
Code-mixed	Northern province school uniform length increase request	1	✓ correct: வடக்கு மாகாண பாடசாலை மாணவிகளின் சீருடையின் நீளத்தை அதிகரிக்க வேண்டும்..
Code-mixed + rules	Northern province school uniform length increase request	1	✓ correct: வடக்கு மாகாண பாடசாலை மாணவிகளின் சீருடையின் நீளத்தை அதிகரிக்க வேண்டும்..

Example 3. Gold document (English translation, shortened): “New Delhi: The Triple Talaq (Prohibition) Bill is being introduced in the Rajya Sabha today (December 31), following which the BJP-Congress …”

Form	Query as given to the retriever	Rank of gold	Top-1 retrieved document (Tamil, shortened)
English	Triple Talaq Bill Rajya Sabha introduction	3	✗ wrong: மேற்கண்ட தகவல்கள்2006 ஜூன் மல்லிகையில் மேமன்கவி அவர்கள் எழுதிய, “எழுத…
Tamil script	முத்தலாக் தடை மசோதா ராஜ்யசபா	1	✓ correct: புதுடில்லி : ‘முத்தலாக்தடை மசோதா, ராஜ்யசபாவில் இன்று(டிச.,31) தாக்கல…
Romanised	Muthalaq thadai masoda Rajyasabha	—	✗ wrong: அழகிய மணிமுடியும்தாமரை மலர் போல சிவந்த திருக்கரங்களும்ஒளிவீசும் வே…
Romanised + rules	முதலக் தடை மசொட ரஜ்யசப	—	✗ wrong: எந்தக் கட்சி ஆட்சியில் இருந்தாலும் ’நான் அடிப்பதுபோல் அடிக்கிறேன்நீ…
Code-mixed	Triple Talaq bill Rajya Sabha la eppo	4	✗ wrong: எந்தக் கட்சி ஆட்சியில் இருந்தாலும் ’நான் அடிப்பதுபோல் அடிக்கிறேன்நீ…
Code-mixed + rules	Triple டலக் bill ரஜ்ய Sabha ல எப்பொ	—	✗ wrong: எந்தக் கட்சி ஆட்சியில் இருந்தாலும் ’நான் அடிப்பதுபோல் அடிக்கிறேன்நீ…

Table 5.9: Example outputs of the dense retriever (multilingual-e5-small) on the Tamil corpus.

Example 3 shows the limits of rule-based transliteration. *Muthalaq* (முத்தலாக் “triple talaq”) becomes முதலக் and *masoda* becomes மசொட instead of மசோதா (“bill”), because the romanised spelling marks neither the doubled *t*, the long *o* nor the final long *a*. Neither the dense model nor the rules recover this query, which points to character-level matching and better transliteration as Module 3 work. The code-mixed version shows a second failure: the language-ID step keeps *Triple*, *bill* and *Sabha* as English but sends *Talaq* and *Rajya* through the Tamil rules, giving “Triple டலக்bill ரஜ்ய Sabha”, which splits one named entity across two scripts.

5.10 Discussion

What the results mean for a RAG builder. A Tamil RAG system that passes Tanglish queries straight to a multilingual retriever misses the correct document for most romanised questions. A transliteration step of a few dozen rules, costing under a millisecond, recovers most of the loss, and a character n-gram index makes it robust to the remaining spelling errors and to Tamil’s agglutinative suffixes, which no stemmer handled here. Hybrid fusion should be gated by script, and zero-score lexical results must never enter the fusion.

Why dense models fail on romanised Tamil. Multilingual embedding models are trained largely on native-script text. Romanised Tamil is split into English-like sub-words with no learned link to the Tamil-script forms. The model does better on code-mixed queries only because those queries are mostly English.

Threats to validity.

1. *Synthetic queries.* Every query was generated by an LLM. Real Tanglish has more Tamil words, more spelling variation and more colloquial grammar. Module 3 tests on human-written queries.
2. *An easy retrieval setting.* Each query was generated from its own document, so lexical overlap with the gold document is higher than in real use. The 500-document corpus is small, and Tamil-script scores are near ceiling.

3. *One relevant document per query.* Other documents could also answer some queries but are counted as wrong, so scores are a lower bound.
4. *One small dense model.* Larger models (BGE-M3, multilingual-e5-base or -large, LaBSE) may behave differently; they are scheduled for Module 3.
5. *Rules written by one author.* The rules were not tuned on test data, but they reflect one person's view of Tanglish spelling. A held-out human set will check that they generalise.
6. *Language ID breaks named entities.* The frequency-threshold language ID (wordfreq Zipf ≥ 3.0) knows English words but not Indian names. Indian named entities written in Roman script, such as *Talaq* or *Rajya*, are judged non-English and transliterated, while their English neighbours are kept, so one entity can end up half in each script (Section 5.9, Example 3). An entity-aware language ID is planned (Section 6.2).
7. *Queries cluster within documents.* The 2,154 queries come from 500 documents (about 4.3 each), so they are not independent. Query-level bootstrap intervals are therefore too narrow. Re-running every key test with a document-cluster bootstrap widens the intervals by about 1.6 times but leaves every conclusion unchanged (Section 5.5).
8. *An earlier fusion bug.* The first run fed zero-score BM25 documents into RRF. It is fixed, and the naive numbers are kept only for comparison (Sections 4.4 and 5.6).

Chapter 6: Conclusion and Future Work

6.1 Conclusion

Modules 1 and 2 defined the problem of code-mixed Tamil–English retrieval for RAG, surveyed 19 related works including the FIRE mixed-script IR tracks and identified a narrower but clear gap, designed a script-aware retrieval pipeline, and implemented and evaluated it on 2,154 queries in four forms with five retrievers and two normalisation methods on a CPU. The main findings:

1. **Romanised Tamil queries break standard retrieval.** Dense retrieval reaches only MRR@10 = 0.143 and the best raw configuration 0.178, against 0.923 for Tamil script.
2. **Transliteration to Tamil script recovers most of the loss cheaply.** Tanglish rules with character-3-gram BM25 reach MRR@10 = 0.684 and Recall@10 = 0.812 at about 0.3 ms normalisation cost, and significantly beat the generic ITRANS scheme.
3. **Naive (equal-weight, ungated) hybrid fusion is harmful across scripts.** With zero-score documents correctly excluded, RRF still cuts dense MRR@10 from 0.788 to 0.710 for English queries on Tamil documents (0.305 with a third, character-3-gram list). An earlier run that fused zero-score documents overstated this as 0.187.
4. **The public code-mixed query set is English-heavy.** 22% of its code-mixed queries contain no Tamil word, so human-written Tanglish data is needed.

6.2 Future modules

Module 3: Human Tanglish questions and Tamil-aware chunking.

- Collect about 500 human-written Tanglish questions from Tamil-speaking students over a longer document collection (for example college circulars, scholarship notices and government notices), with relevance judgements.
- Implement a Tamil-aware chunker that splits at Tamil sentence and clause boundaries and never inside a grapheme cluster, and compare it with the ChunkLab chunkers (fixed-size, recursive, sentence, semantic) at several chunk sizes.
- Make a **script-gated router** the headline system: detect the query's script mix and send it to the retriever that works for that form (dense for English and code-mixed, rules + character-3-gram BM25 for romanised Tamil), fusing only when every list is informative.
- Add baselines: **IndicXlit** neural transliteration [22] in place of the rules, and an **LLM query rewriter** that converts a Tanglish query into Tamil script or English before retrieval.
- Use **longer real documents**, such as Tamil Nadu scholarship notices and Government Orders (G.O.s), so that chunking actually matters.
- Try a **phonetic-skeleton BM25** that indexes consonant skeletons with vowel signs removed and merges ல/ள/ழ, ன/ண/ந and த/ட, to absorb what Tanglish spelling cannot encode (a FIRE 2015 system already removed vowel signs before character n-gram indexing [20]).
- Build an **entity-safe language ID** that keeps Indian named entities (*Rajya Sabha*, *Triple Talaq*) whole, using a gazetteer and capitalisation, instead of a pure English-frequency threshold.

- Add BGE-M3 and LaBSE.

Module 4: Generation-side RAG evaluation. Connect an LLM generator and measure answer correctness and faithfulness for each query form, to test whether retrieval gains carry through to answers.

Module 5: Paper and release. Write up the full factorial study for a workshop or conference (for example FIRE or a code-switching workshop) and release the benchmark and code.

6.3 Timeline

Period	Work
Nov 2026	Module 2 review; design the human query collection protocol
Dec 2026 – Jan 2027	Module 3: collect ~500 Tanglish questions; Tamil-aware chunker; larger embedding models
Feb – Mar 2027	Module 4: LLM generation and answer-quality evaluation
Apr – May 2027	Module 5: paper, thesis write-up, public release

Table 6.1: Planned timeline.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[3] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, 2024.

[4] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv:2402.03216, 2024.

[5] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, "Language-agnostic BERT sentence embedding," in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 878–891.

[6] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[7] B. R. Chakravarthi, V. Muralidaran, R. Priyadharshini, and J. P. McCrae, "Corpus creation for sentiment analysis in code-mixed Tamil-English text," in *Proc. 1st Joint Workshop on Spoken Language Technologies for Under-resourced Languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL)*, 2020, pp. 202–210.

[8] B. R. Chakravarthi, R. Priyadharshini, V. Muralidaran, S. Suryawanshi, N. Jose, E. Sherly, and J. P. McCrae, "Overview of the track on sentiment analysis for Dravidian languages in code-mixed text," in *Proc. 12th Annual Meeting of the Forum for Information Retrieval Evaluation (FIRE '20)*, 2020, pp. 21–24, doi: 10.1145/3441501.3441515.

[9] S. Chanda and S. Pal, "Overview of the shared task on code-mixed information retrieval from social media data," in *FIRE 2024 Working Notes*, CEUR Workshop Proceedings, vol. 4054, 2024, pp. 124–128. [Online]. Available: https://ceur-ws.org/Vol-4054/T2-1.pdf

[10] S. Chanda, K. Tewari, and S. Pal, "Overview of the CMIR track at FIRE 2025: Code-mixed information retrieval from social media data," in *Proc. 17th Annual Meeting of the Forum for Information Retrieval Evaluation (FIRE '25)*, ACM, 2025, pp. 13–16, doi: 10.1145/3777867.3778255. (Author order as in ACM's corrected Version of Record of 27 January 2026; DBLP still lists the original order, Tewari, Chanda and Pal.)

[11] Q. Zeng, Y. Lu, Z. Zhou, H. Qi, P. Yu, F. Zhao, H. Yanaka, W. Xuan, and N. Yokoya, "Code-switching information retrieval: Benchmarks, analysis, and the limits of current retrievers," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, doi: 10.18653/v1/2026.findings-acl.636.

[12] S. PaulRaj K, "ChunkLab: The interactive RAG engineering playground," GitHub repository, 2026. [Online]. Available: https://github.com/selvin-paul-raj/chunklab

[13] K. Chandu, E. Loginova, V. Gupta, J. van Genabith, G. Neumann, M. Chinnakotla, E. Nyberg, and A. W. Black, "Code-mixed question answering challenge: Crowd-sourcing

data and techniques," in *Proc. Third Workshop on Computational Approaches to Linguistic Code-Switching*, Melbourne, Australia, 2018, pp. 29–38, doi: 10.18653/v1/W18-3204.

[14] D. Balasubramaniyam and T. Madushanka, "Query translation vs. cross-lingual embeddings for Sinhala-Tamil e-government information retrieval," arXiv:2608.12820, 2026.

[15] S. K. Singh, "Parser, chunking, and embedding interactions in retrieval-augmented generation over Indian government regulatory documents," arXiv:2609.31660, 2026.

[16] S. Chhoun, P. Po, S. Ros, W.-S. Cho, and S. Khoeurn, "Evaluation of chunking strategies for effective text embedding in low-resource language on agricultural documents," arXiv:2605.22203, 2026.

[17] Madhusudhanan G., "Visible reasoning and indirect prompt-injection monitorability across English, Tamil, and Tanglish," arXiv:2608.15392, 2026.

[18] nlpctx, "Tamil-English code-mixed retrieval evaluation dataset (tamil-english-retrieval)," Hugging Face Datasets, 2026, Apache-2.0. [Online]. Available: https://huggingface.co/datasets/nlpctx/tamil-english-retrieval

[19] S. Banerjee, K. Chakma, S. K. Naskar, A. Das, P. Rosso, S. Bandyopadhyay, and M. Choudhury, "Overview of the mixed script information retrieval (MSIR) at FIRE-2016," in *Working Notes of FIRE 2016*, CEUR Workshop Proceedings, vol. 1737, 2016. [Online]. Available: https://ceur-ws.org/Vol-1737/T3-1.pdf

[20] R. Sequiera, M. Choudhury, P. Gupta, P. Rosso, S. Kumar, S. Banerjee, S. K. Naskar, S. Bandyopadhyay, G. Chittaranjan, A. Das, and K. Chakma, "Overview of FIRE-2015 shared task on mixed script information retrieval," in *Post Proceedings of the FIRE 2015 Workshops*, CEUR Workshop Proceedings, vol. 1587, 2015, pp. 19–25. [Online]. Available: https://ceur-ws.org/Vol-1587/T2-1.pdf

[21] P. Gupta, K. Bali, R. E. Banchs, M. Choudhury, and P. Rosso, "Query expansion for mixed-script information retrieval," in *Proc. 37th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2014, pp. 677–686, doi: 10.1145/2600428.2609622.

[22] Y. Madhani, S. Parthan, P. Bedekar, G. NC, R. Khapra, A. Kunchukuttan, P. Kumar, and M. Khapra, "Aksharantar: Open Indic-language transliteration datasets and models for the next billion users," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, Singapore, 2023, pp. 40–57, doi: 10.18653/v1/2023.findings-emnlp.4.

[23] AI4Bharat, "IndicCMix (code-mixed, native-script and romanised renderings for 11 Indic languages)," Hugging Face Datasets, MIT licence. [Online]. Available: https://huggingface.co/datasets/ai4bharat/IndicCMix

[24] R. Saha Roy, M. Choudhury, P. Majumder, and K. Agarwal, "Overview of the FIRE 2013 track on transliterated search," in *Proc. 5th Forum for Information Retrieval Evaluation (FIRE '13)*, ACM, 2013, pp. 1–7, doi: 10.1145/2701336.2701636.

[25] D. Ganguly, S. Pal, and G. J. F. Jones, "DCU@FIRE-2014: Fuzzy queries with rule-based normalization for mixed script information retrieval," in *Proc. 6th Forum for Information Retrieval Evaluation (FIRE '14)*, ACM, 2014, pp. 80–85, doi: 10.1145/2824864.2824888.

[26] K. Chakma and A. Das, "CMIR: A corpus for evaluation of code mixed information retrieval of Hindi-English tweets," *Computación y Sistemas*, vol. 20, no. 3, 2016, doi: 10.13053/cys-20-3-2459.

[27] R. R. Riyadh and G. Kondrak, “Joint approach to deromanization of code-mixed texts,” in *Proc. 6th Workshop on NLP for Similar Languages, Varieties and Dialects (VarDial)*, Ann Arbor, MI, USA, 2019, doi: 10.18653/v1/W19-1403.

[28] A. Chari, I. Ounis, and S. MacAvaney, “Lost in transliteration: Bridging the script gap in neural IR,” in *Proc. 48th Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR ’25)*, Padua, Italy, 2025, pp. 2900–2905, doi: 10.1145/3726302.3730226.

[29] A. Deroy and S. Maity, “RetrieveGPT: Merging prompts and mathematical models for enhanced code-mixed information retrieval,” in *FIRE 2024 Working Notes*, CEUR Workshop Proceedings, vol. 4054, 2024, [Online]. Available: https://ceur-ws.org/Vol-4054/T2-2.pdf (arXiv:2411.04752).

[30] H. Mishra, R. Sharma, and N. Yadav, “Decoding Benglish: Scalable information retrieval for transliterated code-mixed conversations,” in *FIRE 2025 Working Notes*, CEUR Workshop Proceedings, vol. 4173, 2025, [Online]. Available: https://ceur-ws.org/Vol-4173/T3-9.pdf.

[31] B. Roark, L. Wolf-Sonkin, C. Kirov, S. J. Mielke, C. Johny, I. Demirsahin, and K. Hall, “Processing South Asian languages written in the Latin script: The Dakshina dataset,” in *Proc. 12th Language Resources and Evaluation Conf. (LREC)*, Marseille, France, 2020, pp. 2413–2423.

[32] C. Kirov, C. Johny, A. Katanova, A. Gutkin, and B. Roark, “Context-aware transliteration of romanized South Asian languages,” *Computational Linguistics*, vol. 50, 2024, pp. 475–534, doi: 10.1162/coli_a_00510.

[33] Y. Madhani, M. M. Khapra, and A. Kunchukuttan, “Bhasha-Abhijnaanam: Native-script and romanized language identification for 22 Indic languages,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Toronto, Canada, 2023, doi: 10.18653/v1/2023.acl-short.71.

[34] A. Benton, A. Gutkin, C. Kirov, and B. Roark, “Improving informally romanized language identification,” in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Suzhou, China, 2025, pp. 2318–2336, doi: 10.18653/v1/2025.emnlp-main.117.

[35] H. M. A. D. Herath and T. G. D. K. Sumanathilaka, “TAMZHI: Shorthand romanized Tamil to Tamil reverse transliteration using novel hybrid approach,” *Int. J. Advances in ICT for Emerging Regions (ICTer)*, vol. 17, no. 1, 2024, pp. 1–7, doi: 10.4038/icter.v17i1.7271.

[36] V. Gupta, M. Chinnakotla, and M. Shrivastava, “Transliteration better than translation? Answering code-mixed questions over a knowledge base,” in *Proc. Third Workshop on Computational Approaches to Linguistic Code-Switching*, Melbourne, Australia, 2018, doi: 10.18653/v1/W18-3205.

[37] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, and M. Choudhury, “GLUECoS: An evaluation benchmark for code-switched NLP,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, doi: 10.18653/v1/2020.acl-main.329.

[38] G. Winata, A. F. Aji, Z. X. Yong, and T. Solorio, “The decades progress on code-switching research in NLP: A systematic survey on trends and challenges,” in *Findings of the Association for Computational Linguistics: ACL 2023*, Toronto, Canada, 2023, doi: 10.18653/v1/2023.findings-acl.185.

[39] R. Zhang, S. Cahyawijaya, J. C. B. Cruz, G. Winata, and A. F. Aji, “Multilingual large language models are not (yet) code-switchers,” in *Proc. 2023 Conf. Empirical Methods in*

Natural Language Processing (EMNLP), Singapore, 2023, doi: 10.18653/v1/2023.emnlp-main.774.

[40] G. I. Winata *et al.*, “Can large language models understand, reason about, and generate code-switched text?,” arXiv:2601.07153, 2026.

[41] A. Das, M. Parmar, M. Ramnath, and P. Verma, “Indi-RomCoM: Code-mixed benchmark for evaluating LLMs on romanized Indic-English instructions,” arXiv:2606.30790, 2026.

[42] S. S. Gaikwad and J. S. Gaikwad, “Reducing hallucinations in retrieval-augmented LLM question answering for Indian code-mixed queries,” in *Smart Trends in Computing and Communications (SmartCom 2026)*, Lecture Notes in Networks and Systems, vol. 1991, Cham: Springer, 2026, pp. 402–415, doi: 10.1007/978-3-032-27754-1_37.

[43] S. Sharma, R. Rathee, N. Mishra, and S. Bhatt, “Multilingual embedders for explainable Hinglish retrieval in RAG,” in *Proc. Int. Conf. Artificial Intelligence and Cyber Security (ICAICS 2025)*, Lecture Notes in Networks and Systems, vol. 2024, Cham: Springer, 2026, pp. 157–172, doi: 10.1007/978-3-032-28379-5_13.

[44] T. Harika, Kaniz-E-Fatima, and S. Samreen, “OneVoice: A code-mixed RAG framework for multilingual student support,” *ICTACT J. Soft Computing*, vol. 17, 2026, pp. 4297–4305, doi: 10.21917/ijsc.2026.0591.

[45] A. J. M. Adoptante, J. A. D. V. Castro, M. L. B. Medrana, A. P. B. Ocampo, E. C. Peramo, and M. R. M. Miranda, “Benchmarking open-source large language models on code-switched Tagalog-English retrieval augmented generation,” *J. Advances in Information Technology*, vol. 16, no. 2, 2025, pp. 233–242, doi: 10.12720/jait.16.2.233-242.

[46] P. Agarwal, “Where does noise robustness live? Evidence from ablation, role swaps, and cross-lingual transfer in code-mixed banking NLU,” in *Proc. 11th Workshop on Noisy and User-generated Text (W-NUT)*, 2026, pp. 356–367, doi: 10.18653/2026.wnut-1.26.

[47] S. Doddapaneni, R. Aralikatte, G. Ramesh, S. Goyal, M. M. Khapra, A. Kunchukuttan, and P. Kumar, “Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada, 2023, doi: 10.18653/v1/2023.acl-long.693.

[48] J. P. Gala *et al.*, “IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages,” arXiv:2305.16307, 2023.

[49] A. Balachandran, “Tamil-Llama: A new Tamil language model based on Llama 2,” arXiv:2311.05845, 2023.

[50] R. Kumaresan, Madhusudhanan G., and K. Kezendirane, “Agaram 2.5B: The first Tamil-first language model trained from scratch with tool calling. A playbook for under-represented languages,” arXiv:2610.18420, 2026.

[51] Sarvam AI, “Sarvam-1,” Hugging Face model card, 2024, [Online]. Available: https://huggingface.co/sarvamai/sarvam-1.

[52] S. Haq, A. Sharma, O. Khattab, N. Chhaya, and P. Bhattacharyya, “IndicIRSuite: Multilingual dataset and neural information models for Indian languages,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Bangkok, Thailand, 2024, doi: 10.18653/v1/2024.acl-short.46.

[53] P. Prasanjith, P. More, A. Kunchukuttan, and R. Dabre, “IndicRAGSuite: Large-scale datasets and a benchmark for Indian language RAG systems,” arXiv:2506.01615, 2025.

[54] A. K. Singh, V. Kumar, R. Murthy, J. Sen, A. Mittal, and G. Ramakrishnan, “INDIC QA BENCHMARK: A multilingual benchmark to evaluate question answering capability of LLMs for Indic languages,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, NM, USA, 2025, pp. 2607–2626, doi: 10.18653/v1/2025.findings-naacl.141.

[55] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin, “MIRACL: A multilingual retrieval dataset covering 18 diverse languages,” *Trans. Association for Computational Linguistics*, vol. 11, 2023, doi: 10.1162/tacl_a_00595.

[56] K. Raja, M. Abejide, A. Ram, U. Sharma, and K. Zhu, “MURMUR: Cross-lingual and multimodal retrieval augmented reasoning for open question answering in Tamil and Yoruba,” in *Proc. Conf. Language Modeling (COLM)*, 2026, [Online]. Available: https://proceedings.colm.cc/paper_files/paper/2026/file/2098f348d1d071e3ce652a5edadae3b1-Paper-Conference.pdf.

[57] S. S. Anthea, N. R. Mary, and K. Varalakshmi, “TanglishMedBench: A benchmark for evaluating LLMs on Tamil-English code-mixed medical queries,” Zenodo dataset, v1.0.0, 2026, doi: 10.5281/zenodo.22308075.

[58] P.-A. Duquenne, H. Schwenk, and B. Sagot, “SONAR: Sentence-level multimodal and language-agnostic representations,” arXiv:2308.11466, 2023.

[59] LCM Team, L. Barrault, P.-A. Duquenne, M. Elbayad *et al.*, “Large concept models: Language modeling in a sentence representation space,” arXiv:2412.08821, 2024.

[60] J. A. Husain, R. Dabre, A. Kumar, J. Gala, T. Jayakumar, R. Puduppully, and A. Kunchukuttan, “RomanSetu: Efficiently unlocking multilingual capabilities of large language models via romanization,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, 2024, doi: 10.18653/v1/2024.acl-long.833.

[61] A. A. K. Verma, A. Chatterjee, M. Gupta, and T. Chakraborty, “Multilingual language models encode script over linguistic structure,” in *Proc. 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, San Diego, CA, USA, 2026, pp. 33685–33719, doi: 10.18653/v1/2026.acl-long.1554.

[62] T. Zhu, H. C. Ming, and M.-Y. Kan, “When does mixing help? Analyzing query embedding interpolation in multilingual dense retrieval,” in *Proc. 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, San Diego, CA, USA, 2026, pp. 31544–31562, doi: 10.18653/v1/2026.acl-long.1455.

[63] S. Bruch, S. Gai, and A. Ingber, “An analysis of fusion functions for hybrid retrieval,” *ACM Trans. Information Systems*, vol. 42, 2023, pp. 1–35, doi: 10.1145/3596512.

[64] A. Chauhan and A. Agarwal, “The fusion tax: A diagnostic analysis of when hybrid retrieval hurts,” *IEEE Access*, vol. 14, 2026, pp. 159620–159637, doi: 10.1109/ACCESS.2026.3743502.

[65] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, “Query rewriting in retrieval-augmented large language models,” in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023, doi: 10.18653/v1/2023.emnlp-main.322.

[66] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada, 2023, doi: 10.18653/v1/2023.acl-long.99.

[67] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, “Dense X retrieval: What retrieval granularity should we use?,” in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Miami, FL, USA, 2024, doi: 10.18653/v1/2024.emnlp-main.845.

[68] R. Qu, R. Tu, and F. S. Bao, “Is semantic chunking worth the computational cost?,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, NM, USA, 2025, pp. 2155–2177, doi: 10.18653/v1/2025.findings-naacl.114.

[69] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAs: Automated evaluation of retrieval augmented generation,” in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, St. Julians, Malta, 2024, doi: 10.18653/v1/2024.eacl-demo.16.

[70] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An automated evaluation framework for retrieval-augmented generation systems,” in *Proc. 2024 Conf. North American Chapter of the Association for Computational Linguistics (NAACL), Volume 1: Long Papers*, Mexico City, Mexico, 2024, doi: 10.18653/v1/2024.naacl-long.20.

[71] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, Datasets and Benchmarks Track, 2023, pp. 46595–46623.

[72] R. Hada, V. Gumma, A. de Wynter, H. Diddee, M. Ahmed, M. Choudhury, K. Bali, and S. Sitaram, “Are large language model-based evaluators the solution to scaling up multilingual evaluation?,” in *Findings of the Association for Computational Linguistics: EACL 2024*, St. Julian’s, Malta, 2024, doi: 10.18653/v1/2024.findings-eacl.71.

[73] I. Watts, V. Gumma, A. Yadavalli, V. Seshadri, M. Swaminathan, and S. Sitaram, “PARIKSHA: A large-scale investigation of human-LLM evaluator agreement on multilingual and multi-cultural data,” in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Miami, FL, USA, 2024, pp. 7900–7932.

[74] V. Gumma, A. Raghunath, M. Jain, and S. Sitaram, “HEALTH-PARIKSHA: Assessing RAG models for health chatbots in real-world multilingual settings,” arXiv:2410.13671, 2024.

[75] N. Thakur, L. Bonifacio, C. Zhang, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, B. Chen, M. Rezagholizadeh, and J. Lin, ““Knowing when you don’t know”: A multilingual relevance assessment dataset for robust retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, FL, USA, 2024, doi: 10.18653/v1/2024.findings-emnlp.730.

[76] N. Chirkova, D. Rau, H. Déjean, T. Formal, S. Clinchant, and V. Nikoulina, “Retrieval-augmented generation in multilingual settings,” in *Proc. 1st Workshop on Towards Knowledgeable Language Models (KnowLLM 2024)*, Bangkok, Thailand, 2024, doi: 10.18653/v1/2024.knowllm-1.15.

[77] B. Fetahu, A. Fang, O. Rokhlenko, and S. Malmasi, “Gazetteer enhanced named entity recognition for code-mixed web queries,” in *Proc. 44th Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR ’21)*, 2021, pp. 1677–1681, doi: 10.1145/3404835.3463102.